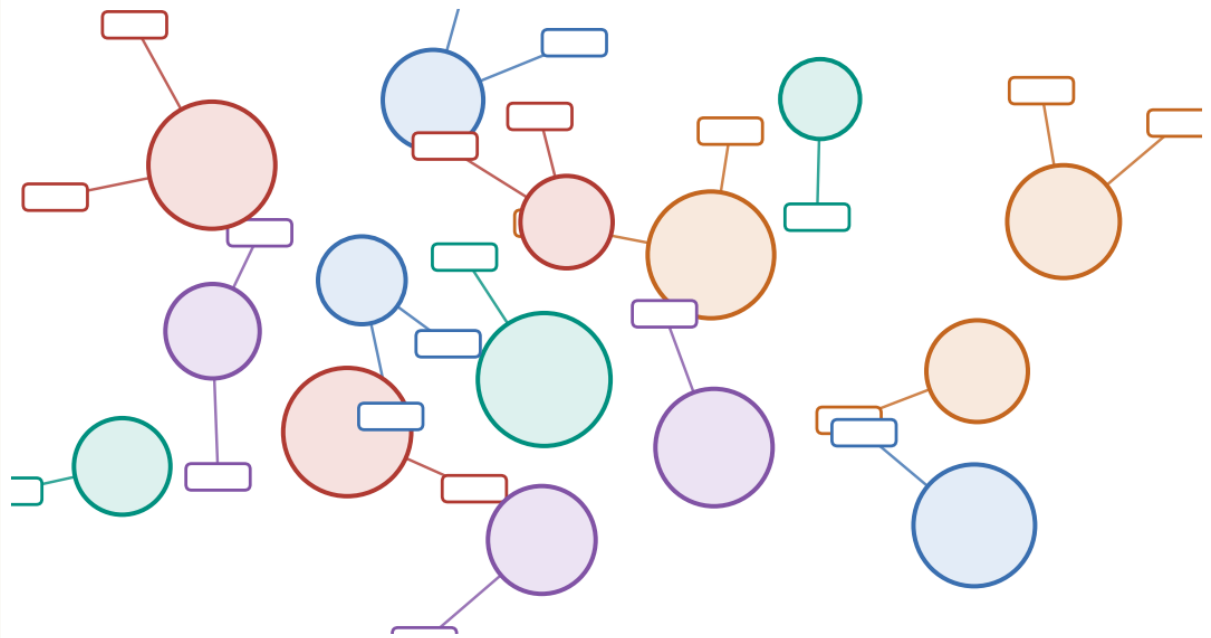
FULL-STACK DATA SCIENCE WITH PYTHON

An illustrated companion to the course notebooks



MODULE 01

Python Foundations for Data Science

The habits that keep your numbers honest

How to read this book

This book walks through every idea in the Module 01 notebooks, but it is written to be *read*, on a train or with a coffee, rather than executed. The notebooks remain the place where you run code, break it and fix it; the book is where you build the mental pictures that make the code obvious.

Each chapter mirrors one notebook and opens with the notebook's file name, so you can always jump between the two. Code in the book is trimmed to the lines that carry the idea, and every figure was generated from code in this repository, using the same colour palette as the notebooks.

Five kinds of boxes appear along the way. They always mean the same thing:

> **NOTE**
> Extra context. Worth knowing, but you can move on without it.

> **TIP**
> A habit or shortcut professionals actually use.

> **WATCH OUT**
> A mistake that produces silently wrong numbers, not an error.

> **FROM THE HISTORY OF DATA SCIENCE**
> A true episode from the history of science and industry that makes the idea stick. Sources are listed at the back.

> **KEY TAKEAWAYS**
> The handful of ideas to carry out of the chapter.

Contents

1 Welcome to the Job . . . 6

1.1 What does a data scientist actually do? . . . 6

1.2 The lifecycle is a loop . . . 8

1.3 Reproducibility starts with the environment . . . 9

1.4 Jupyter without the foot-guns . . . 11

1.5 A first taste of the whole loop . . . 12

1.6 How the repository is organised . . . 14

1.7 The stories in this book . . . 14

1.8 Try it yourself . . . 15

2 Names, Numbers and Other Surprises . . . 16

2.1 Everything is an object . . . 16

2.2 Names are labels, not boxes . . . 17

2.3 Mutability is where the bugs live . . . 18

2.4 Numbers . . . 20

2.5 Strings . . . 22

2.6 Booleans, None and truthiness . . . 23

2.7 Control flow . . . 24

2.8 Try it yourself . . . 25

3 Choosing the Right Container . . . 27

3.1 The map . . . 27

3.2 Lists: the ordered workhorse . . . 28

3.3 Tuples: records, and the hashable sequence . . . 29

3.4 Dictionaries: the most important structure in Python . . . 30

3.5 Sets: membership and deduplication . . . 30

3.6 Why complexity is not academic . . . 31

3.7 Comprehensions . . . 32

3.8 The collections module . . . 33

3.9 Sorting . . . 34

3.10 Unpacking and star expressions . . . 35

3.11 Try it yourself . . . 35

4 Functions That Behave . . . 37

4.1 Anatomy of a good function . . . 37
4.2 Parameters in full . . . 38
4.3 The mutable default argument trap . . . 39
4.4 Scope: the LEGB rule . . . 40
4.5 Decorators . . . 41
4.6 Generators: the memory tool . . . 43
4.7 map, filter, reduce and lambda . . . 44
4.8 itertools: lazy iteration tools . . . 45
4.9 Pure functions and side effects . . . 46
4.10 Try it yourself . . . 46

5 Objects With a Purpose . . . 47

5.1 When should a data scientist write a class? . . . 47
5.2 Classes and instances . . . 48
5.3 Dunder methods . . . 49
5.4 @dataclass: delete the boilerplate . . . 49
5.5 @property: computed and validated attributes . . . 50
5.6 Inheritance versus composition . . . 51
5.7 The scikit-learn estimator pattern . . . 52
5.8 Try it yourself . . . 54

6 Files Without Fear . . . 55

6.1 pathlib: stop building paths with strings . . . 55
6.2 Context managers . . . 56
6.3 A dataset worth measuring . . . 57
6.4 CSV: universal, and full of traps . . . 57
6.5 JSON: nested data . . . 59
6.6 Parquet: the format for data science . . . 59
6.7 A measured comparison . . . 60
6.8 Choosing a format . . . 61
6.9 Try it yourself . . . 61

7 Failing Loudly . . . 63

7.1 The exception hierarchy . . . 63
7.2 Catch narrowly . . . 64
7.3 try / except / else / finally . . . 65
7.4 Custom exceptions . . . 66
7.5 Logging . . . 67

7.6 Defensive validation: fail fast . . . 69
7.7 Testing with pytest . . . 69
7.8 Try it yourself . . . 71

8 Project: A Pipeline You Can Trust . . . 72

8.1 The brief . . . 72
8.2 Step 0: generate the mess . . . 73
8.3 Step 1: write the contract down first . . . 74
8.4 Step 2: domain exceptions . . . 74
8.5 Step 3: extract (the impure shell) . . . 74
8.6 Step 4: transform (the pure core) . . . 75
8.7 Step 5: orchestrate, behind a quality gate . . . 76
8.8 Step 6: the readout . . . 76
8.9 Step 7: test it . . . 77
8.10 Step 8: read back what you wrote . . . 77
8.11 Step 9: prove the safety net fires . . . 77
8.12 Your turn: extensions . . . 77
8.13 What you built . . . 78

Before You Move On . . . 79

Self-assessment . . . 79
Where the exercises and solutions live . . . 79
Further reading . . . 80

Sources for the Stories . . . 81

Chapter 1 . . . 81
Chapter 2 . . . 81
Chapter 3 . . . 81
Chapter 4 . . . 82
Chapter 5 . . . 82
Chapter 6 . . . 82
Chapter 7 . . . 82
Chapter 8 . . . 83

CHAPTER 01

01

Welcome to the Job

What data scientists really do, and the workshop you'll need

Notebook 01_course_introduction_and_setup.ipynb · About 60 minutes

IN THIS CHAPTER

- What a data scientist actually spends the week doing, and which parts this course covers.
- The data science lifecycle, and why it is a loop rather than a pipeline.
- How to build a **reproducible** Python environment, and why that is a professional duty, not a nicety.
- How to work in Jupyter without falling into its two classic traps.
- A first end-to-end taste: data in, insight out, honest conclusion.

Every field has a story it likes to tell about itself. Data science's favourite story is about clever models. The reality is messier, more interesting and, frankly, more useful to know on day one: most of the job is making sure the numbers deserve to be believed.

This first chapter is the map. We'll look at what the work really looks like, set up a workshop that won't betray you, and then run a miniature analysis from start to finish so you can feel the shape of the whole thing before we zoom into the parts.

1.1 What does a data scientist actually do?

Job titles vary wildly, but in a mid-sized or large company the week almost always breaks down into four recurring activities.

Activity	What it looks like	Share of time
Framing	Turning "can we reduce churn?" into a measurable question with a defined success metric	~15%
Data work	Finding, joining, cleaning and validating data; discovering a column changed meaning in March	~50%
Modelling & analysis	Statistics, experiments, machine learning: the part that gets taught	~20%

Activity	What it looks like	Share of time
Delivery	Shipping the model, writing the readout, convincing a stakeholder, monitoring what you shipped	~15%

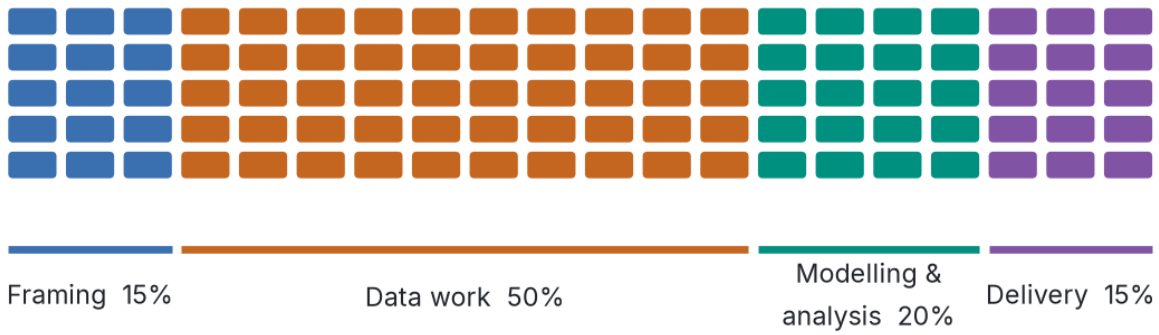


Figure 1.1 Where the week goes. Half of it is data work, and that is not a sign of a badly run team. It *is* the job.

That 50% is the headline. A course that only teaches modelling prepares you for a fifth of your working life. This one tries to cover all four columns.

FROM THE HISTORY OF DATA SCIENCE

The day a job title was invented

Around 2008, two people building data teams at fast-growing web companies, DJ Patil at LinkedIn and Jeff Hammerbacher at Facebook, needed a name for the people they were hiring: part statistician, part engineer, part detective. “Data scientist” stuck. By 2012 the *Harvard Business Review* was calling it “the sexiest job of the 21st century” (Davenport & Patil). The skills were old; the bundle was new, and the bundle is what this course is about.

Three roles, blurry borders

You’ll hear three titles, and their responsibilities overlap more every year:

- **Data Analyst** describes what *happened*: SQL, dashboards, descriptive statistics.
- **Data Scientist** explains *why* it happened and predicts what *will* happen: statistics, experiments, machine learning.
- **ML Engineer** makes the prediction *run reliably* for millions of users: pipelines, serving, monitoring.

The market increasingly expects a data scientist to be credible in all three columns. That is exactly what “full stack” means in this course’s title.

1.2 The lifecycle is a loop

Nearly every project travels through the same eight stages. The single most important thing to internalise is that **the arrows go backwards too.**

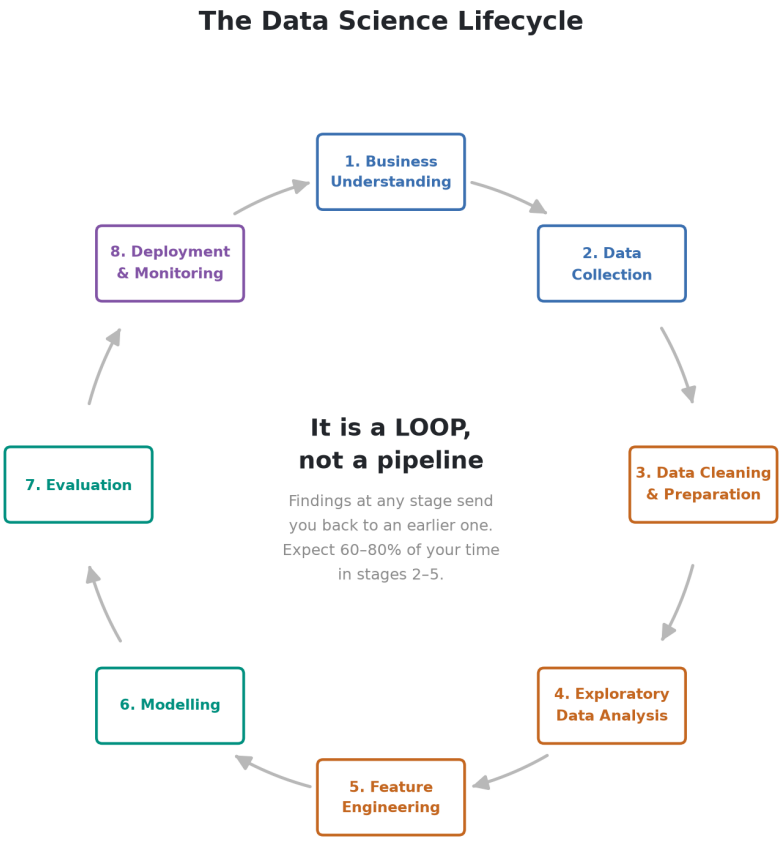


Figure 1.2 The data science lifecycle. Findings at any stage can send you back to an earlier one.

During modelling you'll discover that a feature leaks information from the future, and you'll go back to feature engineering. You'll present results and learn the business question was subtly different, and you'll go back to the very first box. That is not a sign you planned badly. It is the process working.

Stage	The question you're asking	Where the course covers it
1. Business understanding	"What decision will this change?"	Modules 09, 24
2. Data collection	"Where does this live, and can I trust it?"	Modules 05, 20
3. Cleaning & preparation	"What do the missing values *mean*?"	Modules 03, 10
4. Exploratory analysis	"What does the data actually look like?"	Modules 04, 06
5. Feature engineering	"What representation makes the signal learnable?"	Module 10
6. Modelling	"Which model, with which hyperparameters?"	Modules 11–19

Stage	The question you're asking	Where the course covers it
7. Evaluation	"Is it better than what we do today? By how much? How sure am I?"	Modules 08, 13
8. Deployment & monitoring	"Is it still working three months later?"	Modules 21–23

FROM THE HISTORY OF DATA SCIENCE

A map, a pump handle, and the whole loop in 1854

In the summer of 1854 cholera tore through Soho in London, killing hundreds of people in a few streets within days. The physician John Snow did not believe the popular theory that the disease spread through bad air. He **framed** a sharper question (is it the water?), **collected** data by walking door to door recording deaths, **explored** it by plotting them on a street map, and saw them cluster around one public pump on Broad Street. He **delivered** his case to the local council, which removed the pump's handle on 8 September.

Then came the part people forget. The outbreak was already waning when the handle came off, so the intervention alone could not prove his point. Snow went *back around the loop*, comparing death rates between households supplied by two different water companies, and built the stronger argument that eventually changed public health. Question, data, exploration, evaluation, back again: the lifecycle is older than computers.

1.3 Reproducibility starts with the environment

One project, one environment

Imagine two projects on your laptop. Project A was built against an old version of pandas; project B needs a newer one where several behaviours changed. With a single system-wide Python you can only keep one of them happy.

A **virtual environment** is the fix: an isolated folder with its own Python interpreter and its own installed packages. One per project. They are cheap to create, disposable, and they eliminate a whole category of "works on my machine" bugs.

```
full-stack-data-science/
└── .venv/                        ← this project's private environment
    ├── bin/python                ← its own interpreter
    └── lib/python3.12/site-packages/
                                  ← its own packages, invisible to others
```

WATCH OUT

Never `pip install` into your operating system's Python. On macOS and Linux other system tools depend on it, and you can break them.

Three levels of "I wrote down my dependencies"

Professional teams distinguish three levels, and you should too:

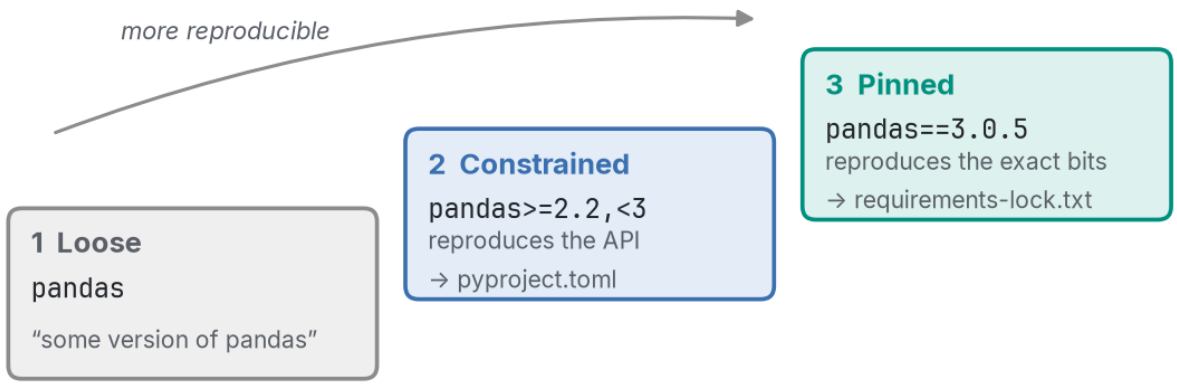


Figure 1.3 The three levels of reproducibility. Humans edit level 2; machines generate level 3; production deploys from level 3.

- **Loose** (`pandas`) means "some version of pandas". It reproduces almost nothing.
- **Constrained** (`pandas>=2.2,<3`) reproduces the *API*. This is what `pyproject.toml` declares.
- **Pinned** (`pandas==3.0.5`) reproduces the *exact bits*. This is what `requirements-lock.txt` captures.

Check what your notebook is really running

The ground truth is not what your terminal says. It is what the *kernel* says:

```
import sys
from pathlib import Path

print(sys.executable)                # the interpreter running THIS notebook
in_venv = sys.prefix != sys.base_prefix
print(f"In a venv? {in_venv}")
```

When `sys.prefix` and `sys.base_prefix` differ, you are inside a virtual environment. If the path points somewhere like `/usr/bin/python3`, your notebook is attached to the wrong kernel. Fix it from the kernel picker before going any further, or nothing later in the course will import.

The course's own helper package

The repository ships a small package, `dstoolkit`, in `src/`. It is installed in **editable mode** (`pip install -e .`), so the import picks up your edits immediately, with no reinstall and no fragile `sys.path` hacks. One call to `setup_plot_style()` then gives every chart in every notebook the same fonts, colours and spacing.

```
from dstoolkit import setup_plot_style
setup_plot_style()
```

FROM THE HISTORY OF DATA SCIENCE

A spreadsheet that shaped economic policy

In 2010 two Harvard economists, Carmen Reinhart and Kenneth Rogoff, published a paper suggesting that economic growth falls sharply once public debt passes 90% of GDP. It was quoted widely in debates about austerity. In 2013 Thomas Herndon, a graduate student at the University of Massachusetts Amherst, tried to reproduce the result for a class assignment and couldn't. When the authors shared their spreadsheet, he found that a formula's cell range stopped short and quietly left five countries out of one average, alongside other debatable choices.

Correcting it weakened the headline finding considerably. The lesson for us isn't "spreadsheets are evil". It is that **analysis nobody else can re-run is analysis nobody can check**, including you, six months from now.

1.4 Jupyter without the foot-guns

A notebook is **not** a script. It is a long-running Python process (the *kernel*) plus a document of cells that you can send to it **in any order**. That freedom is what makes notebooks wonderful for exploration, and it is also the source of both classic notebook bugs.

Trap 1: hidden state

The little counter `[n]` beside each cell tells you the order cells *actually ran*, which may have nothing to do with their order on the page.

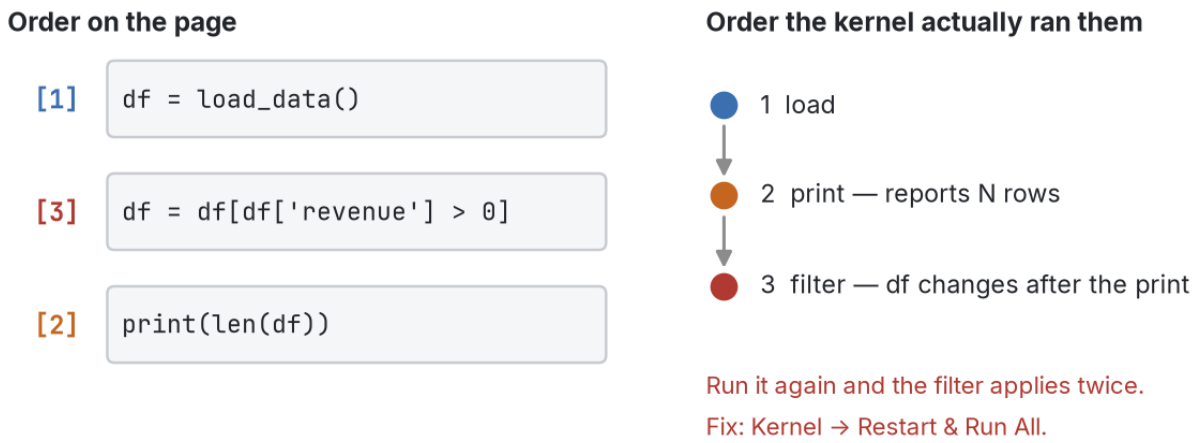


Figure 1.4 Three cells, run out of order. The printed row count describes a DataFrame that no longer exists.

The fix is a habit worth building today: before you trust a result or share a notebook, run **Kernel → Restart Kernel and Run All Cells**. If it fails, your notebook was lying to you.

Trap 2: notebooks are not a substitute for functions

A 200-line cell can't be tested, reused or reviewed. The professional pattern is a **promotion loop**:

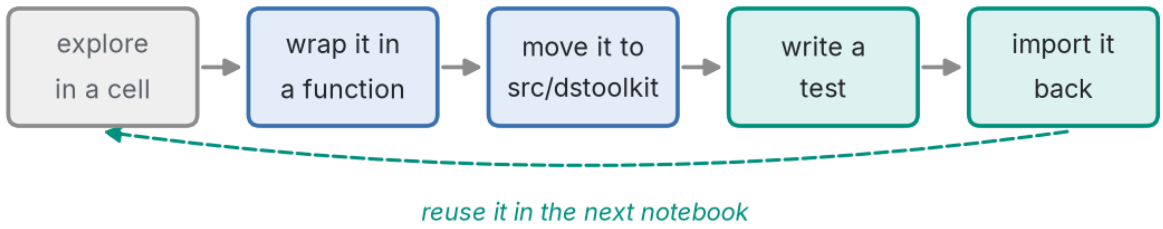


Figure 1.5 The promotion loop: working code graduates from a notebook cell into a tested package, and comes back as an import.

You'll practise this loop for real in this module's project.

The magic commands worth knowing

Jupyter adds "magics": commands prefixed with `%` (one line) or `%%` (the whole cell).

Magic	What it does
`%timeit`	Runs a statement many times and reports a robust average. Use it to choose between two implementations.
`%%time`	Times the whole cell exactly once. For slow, one-off work. Must be the first line of the cell.
`%who` / `%whos`	Lists the variables in the kernel, a quick way to spot hidden state.
`%debug`	Drops you into a post-mortem debugger after an exception.
`%load_ext autoreload` + `%autoreload 2`	Re-imports your `src/` modules automatically after you edit them.
`!command`	Runs a shell command, e.g. `!ls data/raw`.

> **TIP**
> Put `%load_ext autoreload` and `%autoreload 2` at the top of any notebook where you're actively editing code in `src/`. Without them you have to restart the kernel to see your changes.

1.5 A first taste of the whole loop

Enough setup. Let's run the lifecycle in miniature. The scenario comes up in essentially every consumer company: **do customers acquired through paid ads spend less than customers who found us organically?**

We simulate the data so the analysis is self-contained and, thanks to a fixed seed, you'll see exactly the numbers the notebook shows.

```
rng = np.random.default_rng(seed=42)
N_CUSTOMERS = 2_000

channel = rng.choice(["organic", "paid"], size=N_CUSTOMERS, p=[0.55, 0.45])
spend = np.where(
```

```
    channel == "organic",
    rng.lognormal(mean=4.30, sigma=0.55, size=N_CUSTOMERS),
    rng.lognormal(mean=4.05, sigma=0.60, size=N_CUSTOMERS),
)
customers = pd.DataFrame({"customer_id": np.arange(1, N_CUSTOMERS + 1),
                          "channel": channel, "spend_usd": spend.round(2)})
```

Why a *lognormal* distribution? Because real customer spend is right-skewed: most people spend a little, and a few spend a lot. Module 07 explains where that shape comes from.

Look before you leap

The first question about any dataset is always the same: *what's in it, and is any of it obviously broken?* Count rows, count missing values, then summarise by group:

```
summary = customers.groupby("channel")["spend_usd"].agg(
    n="count", mean="mean", median="median", std="std",
)
```

> WATCH OUT
>
> In both groups the **mean sits well above the median**. That is the fingerprint of a right-skewed distribution, and it means the average alone paints a misleading picture of a "typical" customer. Reporting only the mean here is one of the most common analytical mistakes in industry.

A pair of charts (overlapping histograms for the shape, box plots for a like-for-like comparison) shows paid customers sitting a little lower. The notebook draws both.

Is the difference real?

A professional's next question is immediate: *could this gap be noise?* That question, the difference between an observation and a **defensible claim**, is the subject of Modules 07 and 08. Here is a preview using Welch's t-test, which compares two means without assuming the groups share a variance:

```
from scipy import stats

t_stat, p_value = stats.ttest_ind(organic, paid, equal_var=False)
```

> FROM THE HISTORY OF DATA SCIENCE
>
> ### The brewer who signed himself "Student"
>
> The t-test was born in a brewery. William Sealy Gosset worked for Guinness in Dublin, where he needed to draw conclusions from small samples of barley and hops. Guinness didn't want competitors to know it employed statisticians, so when Gosset published his method in 1908 he signed it "Student". That is why, over a century later, you'll still read about "Student's t-distribution".

> **NOTE**
> A small p-value says the gap is unlikely *if* both channels truly spent the same. It does **not** say paid ads *cause* lower spend: the channels differ in many ways at once. Separating correlation from causation is Module 09.

That's the whole loop in one screen: a question, some data, a look at its shape, a quantified comparison, and an honest statement of what it does and does not prove.

1.6 How the repository is organised

```
full-stack-data-science/
├── pyproject.toml              ← dependency declarations (level 2)
├── requirements-lock.txt       ← exact pins (level 3)
├── assets/images/              ← generated diagrams
├── data/
│   ├── raw/                    ← immutable inputs; NEVER edit by hand
│   └── processed/              ← outputs of your notebooks
├── modules/01-python-foundations/
│   ├── notebooks/              ← the lessons
│   ├── scripts/                ← plain-text .py mirrors
│   └── exercises/
├── src/dstoolkit/              ← shared, tested, importable code
└── tests/                      ← pytest suite for src/
```

The one rule about raw data

Raw data is immutable. You never edit it, never overwrite it, never "just fix that one bad row" in Excel. Every transformation is code that reads from `raw/` and writes to `processed/`. If the cleaning lives in code, anyone can re-run it and get your numbers. If it lives in an undocumented manual edit, nobody can, including you.

1.7 The stories in this book

Each chapter includes a short episode from the history of science and industry, chosen because it makes one idea impossible to forget. Here they are on one page; sources are listed at the back of the book.

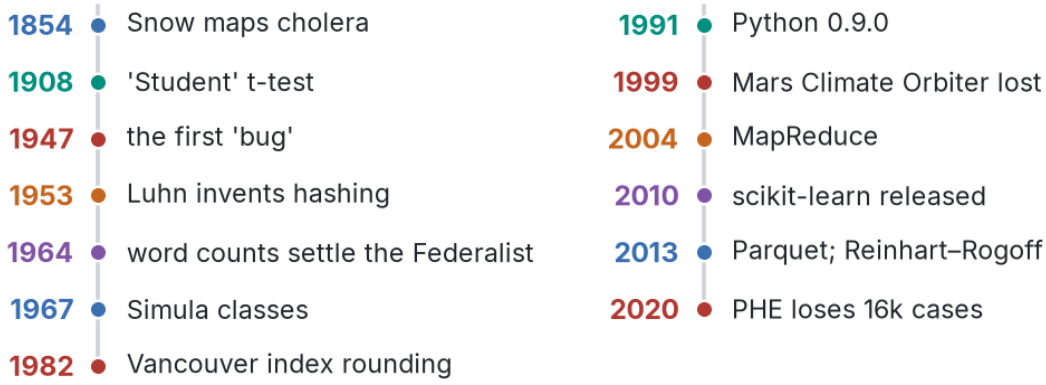


Figure 1.6 The true stories told in this book, in chronological order.

1.8 Try it yourself

These match the exercises at the end of notebook 01; worked solutions are in notebook 09.

1. **Environment audit.** Print the Python version as a `(major, minor, micro)` tuple, whether you're in a virtual environment, and the versions of `numpy`, `pandas` and `matplotlib`. Every package has a `__version__` attribute.
2. **A different seed.** Re-run the customer analysis with `seed=7`. Do the group means change? Does the *conclusion*? What does that tell you about trusting a single sample?
3. **Break the notebook on purpose.** Run all cells, then re-run only the cell that creates `customers` with `N_CUSTOMERS = 50`, and look at `summary`. Explain what happened in one sentence, and name the menu command that prevents it.
4. **Lifecycle mapping.** Pick a data product you use (a recommendation feed, a fraud alert, a delivery ETA) and write one sentence per lifecycle stage about what its team probably had to do.

KEY TAKEAWAYS

- Data science is about **half data work.** The modelling is the small, visible tip.
- The lifecycle is a **loop**: going backwards is the process working.
- **One virtual environment per project.** Declare loose constraints for humans; generate exact pins for machines.
- A notebook is a long-running process. **Restart & Run All** before you trust or share a result.
- Promote working code out of notebooks into a tested package.
- **Raw data is immutable.** Every transformation is code.

CHAPTER 02

02

Names, Numbers and Other Surprises

The object model, and why b = a never copies anything

Notebook 02_python_fundamentals.ipynb · About 90 minutes

IN THIS CHAPTER

- Python's object model, and why `b = a` does not copy anything.
- Mutability and aliasing, the silent bug behind most data-pipeline mysteries.
- Integers that never overflow, floats that never quite add up, and `Decimal` for money.
- Strings, f-strings and text encodings.
- Truthiness, `None`, and the three different kinds of "missing".
- Control flow, including `for...else` and structural pattern matching.

If you only read one chapter of this module carefully, make it this one. Nearly every subtle bug you'll ever write in pandas traces back to something on these pages, usually to the very first idea: what a variable actually *is*.

2.1 Everything is an object

Python has no "primitive types" that live by different rules. An integer, a string, a function, a class, even a module: all of them are objects, and every object carries exactly three things.

Property	How you see it	Can it change?
Identity, its address in memory	`id(obj)`	Never
Type, what kind of thing it is	`type(obj)`	Never
Value, what it holds	the object itself	*Only if the type is mutable*

Functions are objects too, which is why you can hand one to a new name without calling it:

```
def greet(name: str) -> str:
    return f"Hello, {name}"

say_hi = greet          # no call, no parentheses: a second name, same function
say_hi("again")         # 'Hello, again'
```

2.2 Names are labels, not boxes

Here is the most important idea in the chapter.

If you learned C or Java first, you probably picture a variable as a **box that holds a value**. Python doesn't work like that. A name is a **label tied to an object**, and assignment moves the label. It never copies the object.

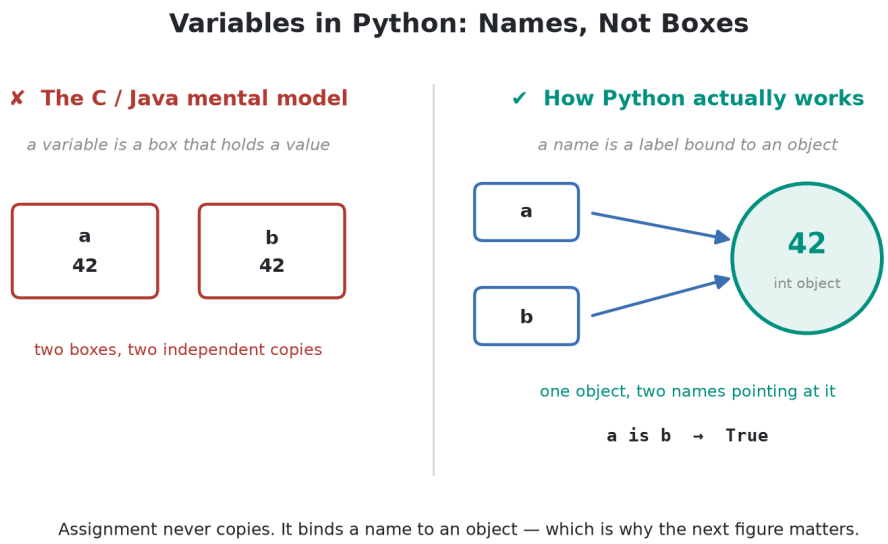


Figure 2.1 Two mental models. In Python, assignment ties another label to the same object.

Think of a cloakroom: you hand in your coat and get a numbered ticket. The ticket isn't the coat. Give a friend a photocopy of your ticket and you now have two tickets for *one* coat. If your friend takes the coat and sews a patch on it, you'll find the patch too.

```
a = [1, 2, 3]
b = a               # a second ticket for the SAME list
print(a is b)       # True: same object
```

`==` versus `is`

These two operators ask genuinely different questions:

- `a == b` asks: do these have the same **value**?
- `a is b` asks: are these the **same object**?

```
list_1 = [1, 2, 3]
list_2 = [1, 2, 3]      # a separate list that happens to hold equal values
list_1 == list_2        # True: same contents
list_1 is list_2        # False: two different objects
```

WATCH OUT

Use `is` only for `None`, `True` and `False`. For everything else, use `==`. CPython pre-builds small integers (−5 to 256) and shares them, so `100 is 100` happens to be `True` while `1000 is 1000` may be `False`. That is an implementation detail you must never rely on.

2.3 Mutability is where the bugs live

Every type is either **mutable** (it can change in place) or **immutable** (it can't). That one property decides whether `b = a` is harmless or a time bomb.

Immutable	Mutable
`int`, `float`, `bool`, `complex`	`list`
`str`	`dict`
`tuple`	`set`
`frozenset`, `bytes`	`bytearray`, `numpy.ndarray`, `pandas.DataFrame`

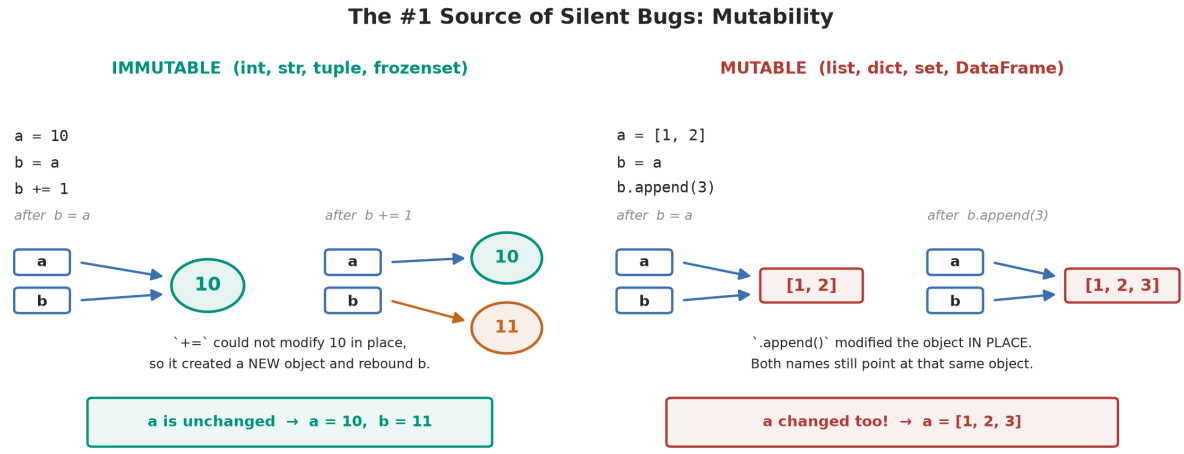


Figure 2.2 Mutable versus immutable: "changing" an immutable value really creates a new object.

```
a = 10
b = a
b += 1          # ints can't change in place → a NEW int; b's label moves
                # a is still 10

x = [1, 2]
y = x
y.append(3)     # lists CAN change in place → both labels see it
                # x is now [1, 2, 3], even though we only touched y
```

Why this matters in real data work

The toy example looks easy to avoid. It isn't, because in real code the aliasing hides behind a function call:

```python
def add_engineered_feature(records: list[dict]) -> list[dict]:
    """BUG: mutates the caller's data as a side effect."""
    for record in records:
        record["total"] = record["price"] * record["quantity"]
    return records

enriched = add_engineered_feature(raw_orders)
# raw_orders has silently gained a "total" key: our "raw" data is gone,
# and enriched is raw_orders → True
```

The fix: copy, or better, don't mutate

Python has two kinds of copy, and the difference shows up as soon as your data is nested, which real data always is.

Copy	Syntax	New container?	New contents?
Shallow	`list(x)`, `x[:]`, `copy.copy(x)`	✔	✘, shares the inner objects
Deep	`copy.deepcopy(x)`	✔	✔, copies all the way down

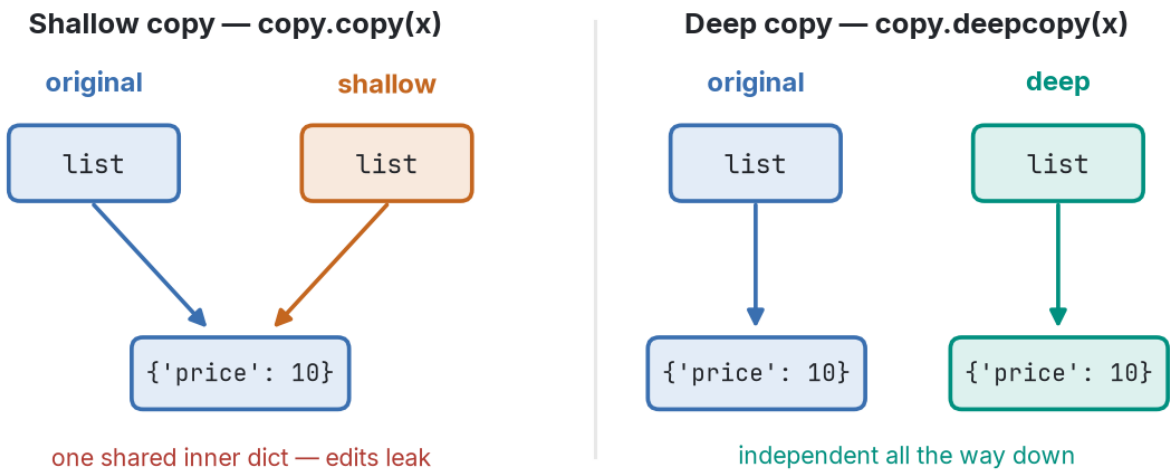


Figure 2.3 A shallow copy duplicates only the outer container. A deep copy duplicates everything inside it too.

Copying defensively everywhere gets tiring, though. The professional habit is to write functions that **don't mutate their inputs at all.** Build and return something new:

```python
def add_total(records: list[dict]) -> list[dict]:
    """Return new records with a `total` field. The input is never touched."""
    return [
        {**record, "total": record["price"] * record["quantity"]}
        for record in records
    ]
```

`{**record, "total": ...}` builds a brand-new dict from the old one plus one key.

> **TIP**
> This is exactly why most pandas methods return a new object rather than editing in place, and why `inplace=True` is discouraged in modern pandas. You'll meet the idea again in Module 03.

2.4 Numbers

Integers never overflow (in Python)

Python integers grow to whatever size they need. There's no ceiling:

```
2**200          # a 61-digit number, no problem
population = 8_100_000_000    # underscores are just for readability
```

WATCH OUT

NumPy integers do not behave like this. For speed they use fixed widths such as `int64`, and they wrap around silently when they run out of room. Doubling `np.int64(2**62)` lands one step past the ceiling and comes back as the most negative number the type can hold. A revenue total that silently turns negative is the kind of bug that reaches a dashboard before anyone notices. Module 02 takes this apart properly.

FROM THE HISTORY OF DATA SCIENCE

The video that broke a counter

In December 2014, the music video for *Gangnam Style* became the first YouTube video to pass 2,147,483,647 views, the largest number a signed 32-bit integer can hold. YouTube had already switched its view counter to 64 bits, and said so with a joke on its own page. It's a light-hearted example of a very serious rule: every fixed-width number has a ceiling, and **someone has to decide in advance what happens when you reach it.**

Floats are approximations

Floats are stored in binary, following the IEEE 754 standard. Just as 1/3 can't be written exactly in decimal (0.3333...), **0.1 can't be written exactly in binary.** Every float therefore carries a tiny representation error, and arithmetic can nudge it one representable step sideways.

```
0.1 + 0.2               # 0.30000000000000004
0.1 + 0.2 == 0.3        # False
f"{0.1:.20f}"           # '0.10000000000000000555'
```

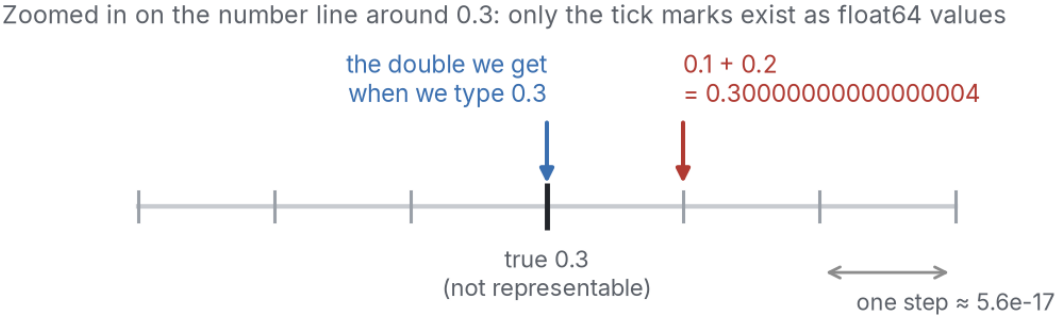


Figure 2.4 The number line near 0.3, magnified about 10^{17} times. Only the tick marks exist as float64 values; 0.1 + 0.2 lands one tick away from the float you get by typing 0.3.

Comparing floats correctly

Never use `==` on floats you computed. Compare within a tolerance:

```
import math
import numpy as np

math.isclose(0.1 + 0.2, 0.3)                  # True
np.isclose(0.1 + 0.2, 0.3)                    # True, and it works on arrays
np.allclose(np.array([0.1 + 0.2, 3.0]), np.array([0.3, 3.0]))   # True
```

When exactness is mandatory: money

For financial calculations, accumulated float error is unacceptable. Auditors don't accept "it was off by 10^{-16} per row, times forty million rows". Use `decimal.Decimal`, which does exact base-10 arithmetic:

```
from decimal import Decimal

Decimal("0.10") + Decimal("0.20") == Decimal("0.30")   # True
Decimal(0.1)     # 0.1000000000000000055511151231257827…  ← the trap!
```

Always build a `Decimal` from a **string**. `Decimal(0.1)` faithfully copies the float's error, which defeats the whole point.

> FROM THE HISTORY OF DATA SCIENCE
>
> **The stock index that lost half its value to rounding**
>
> In January 1982 the Vancouver Stock Exchange launched a new index at 1000.000. Twenty-two months later it stood at about 520, even though the market it tracked had broadly been rising. The culprit: the index was recalculated thousands of times a day, and each time the result was **truncated** (chopped) to three decimal places instead of rounded. Each chop shaved off a fraction of a thousandth, always downwards. When the exchange recomputed the index properly in November 1983, it jumped overnight from roughly 524 to roughly 1099. Tiny, systematic numeric errors don't average out. They **accumulate**.

Division, floor division and negative modulo

```
7 / 2      # 3.5   true division, always a float
7 // 2     # 3     floor division
7 % 2      # 1     remainder
-7 // 2    # -4    floors DOWN, toward minus infinity (C and Java give -3)
-7 % 2     # 1     the sign follows the divisor
```

That last rule surprises people, but it makes cyclical features behave: hour 26 wraps to `26 % 24 == 2`, and hour −3 wraps to `-3 % 24 == 21`.

2.5 Strings

Strings are **immutable sequences of Unicode characters.** Every "change" makes a new string:

```
s = "data science"
s.upper()      # 'DATA SCIENCE', a NEW string; s is unchanged
s[0], s[-1]    # 'd', 'e'   negative indexes count from the end
s[0:4]         # 'data'     [start:stop], stop is excluded
s[::-1]        # 'ecneics atad'
```

The cleaning toolkit

Cleaning text columns is a daily chore, and a handful of methods cover most of it:

```
messy = "  Sao Paulo, BR \n"
messy.strip()                    # 'Sao Paulo, BR'
messy.strip().lower()            # 'sao paulo, br'
messy.strip().split(",")         # ['Sao Paulo', ' BR']
messy.strip().startswith("Sao")  # True
"-".join(["2024", "01", "15"])   # '2024-01-15'
```

> **TIP**
> Use `"".join(parts)` to glue many strings together. Building a string with `+=` in a loop is $O(n^2)$, because every step allocates a whole new string.

f-strings: formatting you should use everywhere

The mini-language after the colon removes a surprising amount of manual rounding code:

```
value, ratio, name = 1234567.8915, 0.0734, "revenue"
f"{value:,.2f}"     # '1,234,567.89'   thousands separator, 2 decimals
f"{ratio:.1%}"      # '7.3%'
f"{value:.3e}"      # '1.235e+06'
f"{name:>15}"       # right-aligned in 15 characters
f"{42:05d}"         # '00042'
n_rows = 1500
f"{n_rows=}"        # 'n_rows=1500'  ← brilliant for debugging
```

Encoding: half of your data-loading errors

A `str` is abstract text. To write it to disk or send it over a network it must be **encoded** into bytes. UTF-8 is the right default; legacy files are often Latin-1 or Windows-1252, and decoding with the wrong one corrupts your data.

```
"café".encode("utf-8")      # b'caf\xc3\xa9'   (5 bytes)
"café".encode("latin-1")    # b'caf\xe9'       (4 bytes)
"café".encode("utf-8").decode("latin-1")   # 'cafÃ©'  ← mojibake
```

NOTE

That garbled `cafÃ©` has a name: **mojibake**, from the Japanese for "character transformation". Japanese computer users met it constantly in the days of several competing encodings, so they named it first.

TIP

When `pd.read_csv` raises a `UnicodeDecodeError`, the file isn't broken. It simply isn't UTF-8. Try `encoding="latin-1"` or `encoding="cp1252"`. Chapter 6 covers this properly.

2.6 Booleans, `None` and truthiness

Every object can be used where Python expects true or false. The rules are short:

- **Falsy:** `False`, `None`, `0`, `0.0`, `""`, `[]`, `{}`, `set()`, `()`.
- **Truthy:** everything else, including `"False"`, `[0]` and `-1`.

So the idiomatic emptiness check is simply `if not items:`, with no `len(...) == 0` needed.

WATCH OUT

This idiom is one of the few places where **NumPy and pandas deliberately break from plain Python.** Asking whether a multi-element array is "true" is ambiguous (any element? all of them?), so `if arr:` raises a `ValueError`. Say what you mean: `arr.any()`, `arr.all()`, or `arr.size` to test for emptiness.

Three kinds of "missing"

In data work you'll meet three different nulls, and they are not interchangeable:

Value	What it is	Equal to itself?
`None`	Python's null object	`None == None` → `True`
`np.nan`	IEEE 754 "not a number", a **float**	`np.nan == np.nan` → `False`, always
`pd.NaT`	pandas' missing timestamp	behaves like `nan`

NaN's defining property really is that it isn't equal to itself. Test for it with `np.isnan(x)` or `.isna()`, never with `==`.

WATCH OUT

`nan != nan` is why `df[df["col"] == np.nan]` silently returns **zero rows** instead of your missing values. The correct call is `df[df["col"].isna()]`. This mistake makes it into production regularly.

2.7 Control flow

Chained comparisons

Python lets you write range checks the way a mathematician would, and evaluates the middle term only once:

```
def classify_spend(amount: float) -> str:
    if amount < 0:
        return "invalid"
    elif 0 <= amount < 50:
        return "low"
    elif 50 <= amount < 200:
        return "medium"
    return "high"
```

Loop over things, not indexes

The surest sign of someone arriving from another language is `for i in range(len(x))`. Python can iterate the object directly, and gives you helpers for the two common extras:

```
for channel in channels:                          # just the items
    ...
for i, channel in enumerate(channels, start=1): # items plus a counter
    ...
for channel, revenue in zip(channels, revenues, strict=True):
    ...                                            # two sequences in step
```

`strict=True` (Python 3.10+) raises if the lengths differ, instead of silently dropping the extras. Silent truncation has hidden a great many data-alignment bugs.

`for...else`

A genuinely useful feature almost nobody knows: the `else` block of a loop runs **only if the loop finished without hitting** `break`. It reads naturally as "if we never found it":

```
for value in values:
    if value < 0:
        print(f"found a negative value: {value}")
        break
else:
    print("no negative values: data passes validation")
```

Structural pattern matching

Added in Python 3.10, `match` is much more than a `switch` statement: it checks the **shape** of data and pulls pieces out in the same breath. It's excellent for messy API responses:

```
match response:
    case {"status": "ok", "data": [*items]}:
        return f"success with {len(items)} records"
    case {"status": "ok"}:
        return "success, no data returned"
    case {"status": "error", "code": int(code)} if code >= 500:
        return f"server error ({code}): safe to retry"
```

```
    case {"status": "error", "code": int(code)}:
        return f"client error ({code}): do not retry"
    case _:
        return "unrecognised response shape"
```

The walrus operator `:=`

`:=` assigns *and* returns a value inside an expression, which saves computing something twice:

```
for reading in readings:
    if (scaled := reading * 1.8 + 32) > 100:
        print(f"{reading} → {scaled:.1f} above threshold")
```

FROM THE HISTORY OF DATA SCIENCE

Why it's called Python

Over the Christmas holidays of 1989, the Dutch programmer Guido van Rossum started a hobby project to keep himself busy: a scripting language that would be pleasant to read. He named it after the BBC comedy *Monty Python's Flying Circus*, not the snake. Version 0.9.0 appeared in February 1991. Readability was the point from day one, which is why so much of this chapter is about writing code that says what it means.

2.8 Try it yourself

Exercises 2.1 to 2.5 in notebook 02; worked solutions are in notebook 09.

1. **Predict before you run.** Write down the output first, then check. Why do (a) and (b) differ, and why does (c) work even though tuples are immutable?

```
a = [1, 2, 3]; b = a; b = b + [4]; print(a)    # (a)
a = [1, 2, 3]; b = a; b += [4];     print(a)    # (b)
t = ([1, 2], 3); t[0].append(99);   print(t)    # (c)
```

1. **A float comparison bug.** `is_fully_allocated([0.1, 0.2, 0.3, 0.4])` should be `True` but returns `False` because it checks `sum(weights) == 1.0`. Fix it.
2. **Safe enrichment.** Write `add_discount(orders, pct)` that returns new dicts with a `discounted_price` key, and prove with an assertion that the input list is unchanged.
3. **Text cleaning.** Write `clean_city(raw)` that turns `" sÃO paulo\n"` into `"Sao Paulo"`. Hint: `" ".join(raw.split())` collapses whitespace in one step.
4. **Missing-value audit.** Given `[1.0, np.nan, 3.0, None, 5.0]`, count the missing entries, and explain why `values.count(np.nan)` gives the wrong answer.

KEY TAKEAWAYS

- **Names are labels tied to objects.** Assignment never copies.
- **Mutability decides everything.** Functions that mutate their arguments create bugs invisible at the call site, so return new objects instead.
- `copy.copy` is shallow; nested data needs `copy.deepcopy`.
- **Floats are approximate.** Compare with `math.isclose` / `np.isclose`; use `Decimal` (built from strings) for money.
- Python ints are unbounded; **NumPy ints overflow silently**.
- A `UnicodeDecodeError` means "wrong codec", not "broken file". UTF-8 is the default.
- `nan != nan`: test missing values with `.isna()` or `np.isnan`.
- Iterate over objects, not indexes: `enumerate`, `zip(..., strict=True)`.

CHAPTER 03

03

Choosing the Right Container

Pick by complexity, not by habit

Notebook 03_data_structures.ipynb · About 90 minutes

IN THIS CHAPTER

- The four built-in collections and how to choose between them **by complexity**.
- Why `x in my_set` can be thousands of times faster than `x in my_list`, measured rather than asserted.
- The `collections` module: `Counter`, `defaultdict`, `deque`, `namedtuple`.
- Comprehensions, the idiom that replaces most of your `for` loops.
- Sorting with key functions, and why a **stable** sort matters.
- Unpacking and star expressions.

A kitchen has drawers, shelves, a spice rack and a fridge. You could keep everything in one big drawer, and for a while it would even work. Then one day you need the cumin in a hurry. Python's built-in collections are the same: each one is shaped for a particular kind of reaching-in, and choosing the wrong one is the commonest cause of slow Python.

3.1 The map

Here's the summary to carry around. Almost every everyday performance problem in Python comes from misreading the "membership" row.

Choosing a Built-in Collection

	list	tuple	dict	set
Ordered	yes	yes	yes (insertion)	no
Mutable	yes	no	yes	yes
Duplicates	yes	yes	keys: no	no
Membership `x in c`	O(n)	O(n)	O(1)	O(1)
Index / key access	O(1)	O(1)	O(1)	n/a
Append / add	O(1)*	n/a	O(1)	O(1)
Can be a dict key	no	yes**	no	no
Literal syntax	`[1, 2]`	`(1, 2)`	`{'a': 1}`	`{1, 2}`

* amortised — an occasional resize costs O(n) ** only if every element it contains is itself hashable

Rule of thumb: reach for a set or dict the moment you find yourself writing `if x in my_list` inside a loop.

Figure 3.1 Choosing a built-in collection: ordering, mutability, duplicates and, above all, the cost of asking "is x in here?".

3.2 Lists: the ordered workhorse

A `list` is a mutable, ordered sequence. Internally it's an **array of pointers** to objects, and that one fact explains its whole personality: indexing by position is instant, but finding a *value* means walking the list.

```
revenues = [1200.50, 980.00, 1540.75, 620.25, 2100.00]
len(revenues), sum(revenues), min(revenues), max(revenues)
```

Slicing: `[start:stop:step]`

The `stop` index is always **excluded**. That isn't an accident. It makes `len(x[a:b]) == b - a`, and it makes `x[:n] + x[n:] == x` true for every `n`.

```
data = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
data[2:5]     # [2, 3, 4]
data[:3]      # [0, 1, 2]       the first three
data[-3:]     # [7, 8, 9]       the last three
data[::2]     # [0, 2, 4, 6, 8] every second element
data[::-1]    # reversed
data[:]       # a NEW list: the standard shallow-copy idiom
```

FROM THE HISTORY OF DATA SCIENCE

Why counting starts at zero

In 1982 the computer scientist Edsger Dijkstra wrote a short handwritten note, *Why numbering should start at zero* (EWD831). His argument was about exactly this slicing convention: describing a range as "from a, up to but not including b" makes lengths a simple subtraction and lets adjacent ranges meet without overlapping or leaving gaps. Start counting at zero and the first n items are simply `[0, n)`. Python, NumPy and pandas all follow his advice.

The methods worth knowing

```
demo = [1, 2, 3]
demo.append(4)        # add one item at the end          O(1)
demo.extend([5, 6])   # add many                         O(k)
demo.insert(0, 0)     # insert at a position             O(n), shifts everything
demo.remove(3)        # delete the first matching VALUE  O(n)
demo.pop()            # remove and return the last item  O(1)
```

WATCH OUT

Mutating methods return `None`, not the list. `sorted_list = my_list.sort()` stores `None`, a slip almost everyone makes once. Use `sorted(my_list)` when you want a new sorted list and the original left alone.

3.3 Tuples: records, and the hashable sequence

Tuples serve two different purposes, and mixing them up causes confusion:

1. **Records:** a fixed set of fields where *position has meaning*, like `(lat, lon)`.
2. **Immutability:** a sequence that can be a `dict` key or a `set` member.

```
latitude, longitude = (-23.55, -46.63)       # unpacking reads a record

city_populations = {
    ("Brazil", "Sao Paulo"): 12_400_000,      # a composite key
    ("Japan", "Tokyo"): 13_960_000,
}
{["Brazil", "Sao Paulo"]: 1}   # TypeError: unhashable type: 'list'
```

NOTE

Immutable doesn't mean deep-frozen. A tuple promises its *references* never change; the objects they point to may still be mutable. So `t = ([1, 2], "fixed"); t[0].append(3)` works, while `t[0] = [9, 9]` raises.

3.4 Dictionaries: the most important structure in Python

A `dict` maps **hashable keys** to values, with **O(1)** average lookup, and since Python 3.7 it remembers insertion order. Loop over pairs with `.items()`:

```
customer = {"id": 1042, "name": "Ana Silva", "channel": "organic"}
for key, value in customer.items():
    print(key, value)
```

Safe access

Indexing a missing key raises `KeyError`. When a field may simply be absent from some records, ask politely instead:

```
record = {"id": 7, "spend": 100.0}
record.get("channel")                 # None
record.get("channel", "unknown")      # 'unknown'
record.setdefault("channel", "unknown")   # inserts the default if absent
```

Merging

The `|` operator (Python 3.9+) merges two dicts; keys on the right win. That makes it perfect for layering settings over defaults, and neither input is modified:

```
defaults = {"model": "linear", "alpha": 0.1, "max_iter": 100}
config = defaults | {"alpha": 0.5}    # {'model': 'linear', 'alpha': 0.5, ...}
{**defaults, "alpha": 0.5}            # the same, as unpacking syntax
```

3.5 Sets: membership and deduplication

A `set` is an unordered collection of unique, hashable elements with **O(1)** membership tests. It's the right answer far more often than people realise:

```
set([101, 102, 101, 103, 102, 104])     # {101, 102, 103, 104}
```

Set algebra

Set operators replace a lot of loop-and-compare code, and they map directly onto the SQL joins you'll meet in Module 05:

```
last_month = {"ana", "bruno", "carla", "diego"}
this_month = {"bruno", "carla", "elena", "felipe"}

last_month & this_month   # retained   {'bruno', 'carla'}
last_month - this_month   # churned    {'ana', 'diego'}
this_month - last_month   # new        {'elena', 'felipe'}
last_month | this_month   # everyone
last_month ^ this_month   # in exactly one month
```

3.6 Why complexity is not academic

Here's the heart of the chapter. To answer `x in container`, a list has to look at elements one by one until it finds a match or runs out: that's **O(n)**. A set runs the value through a **hash function**, which points straight at the one bucket where it would live: that's **O(1)**.

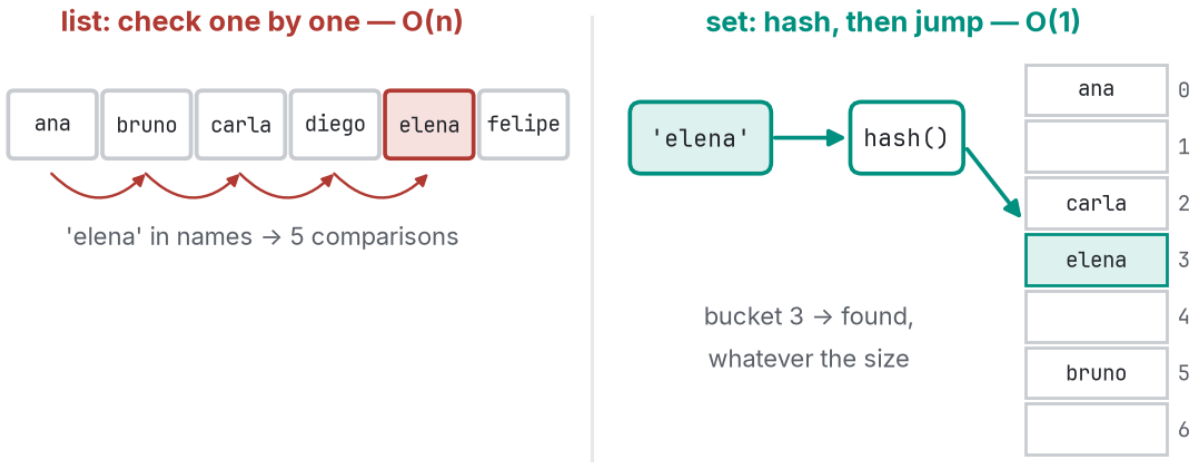


Figure 3.2 A list checks every element in turn. A set hashes the value and jumps to the right bucket, however big the set is.

FROM THE HISTORY OF DATA SCIENCE

A memo at IBM, 1953

The trick behind every set and dict in Python is older than most programming languages. In 1953 Hans Peter Luhn, an engineer at IBM, wrote an internal memo proposing to turn a key into a number and use that number to choose a "bucket" to look in, rather than searching everything. Luhn also gave his name to the check-digit formula still used to validate credit card numbers. Seventy years later, his idea is what makes `if user_id in seen:` instant.

The notebook times a batch of lookups against containers of growing size, choosing values that *aren't* present (the worst case for a list, which must check everything before saying no):

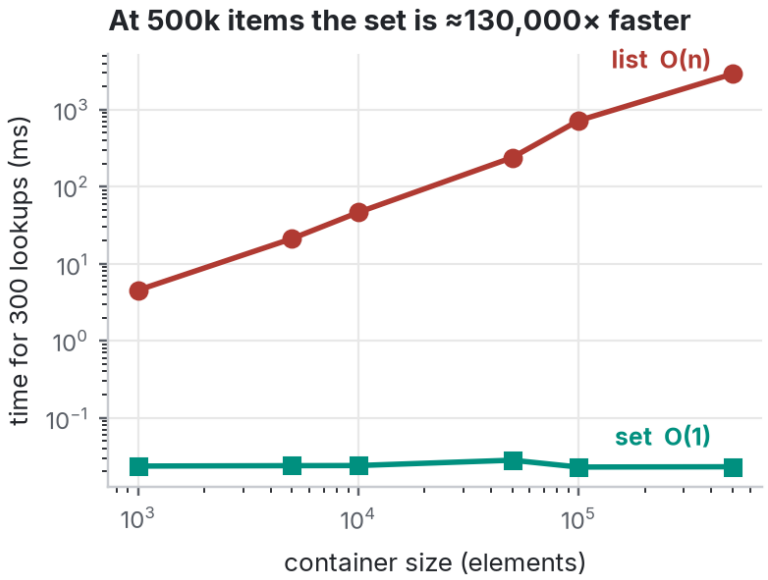


Figure 3.3 Measured on the machine that built this book. Both axes are logarithmic: the list's cost climbs with every extra element while the set's stays flat.

TIP

The practical rule: the moment you write `if x in some_list` *inside a loop*, convert the list to a set first. One extra line routinely turns a multi-hour job into a multi-second one.

```
valid = set(load_valid_ids())          # O(n + m) instead of O(n × m)
clean = [r for r in records if r["id"] in valid]
```

The trade-off: memory

Sets and dicts buy speed with space: their hash tables are deliberately kept roughly a third empty so collisions stay rare. For 100,000 integers, a set takes several times the memory of the equivalent list. Usually a great trade; worth knowing when memory is the limit.

3.7 Comprehensions

A comprehension builds a collection from an iterable in a single expression. It's more readable than the equivalent loop, and faster too, because the looping happens inside the interpreter's C code.



Figure 3.4 The three parts of a comprehension, read left to right.

```
temperatures_c = [18.5, 22.0, 31.2, 15.8, 28.4, 35.1]
warm_f = [t * 9 / 5 + 32 for t in temperatures_c if t > 20]
```

Dict and set comprehensions follow the same pattern:

```
catalogue = {name: price for name, price in zip(products, prices, strict=True)}
premium = {n: p * 0.9 for n, p in catalogue.items() if p > 100}
initials = {name[0] for name in products}        # deduplicates for free
```

Filter or transform?

Position matters. An `if` at the **end** *removes* items; an `if/else` at the **front** *transforms* every item:

```
values = [10, -5, 8, -3, 0]
[v for v in values if v > 0]          # [10, 8]          drops items
[v if v > 0 else 0 for v in values]   # [10, 0, 8, 0, 0] keeps all, clips them
```

WATCH OUT

Don't nest more than two levels. If a comprehension needs a comment to be understood, write the loop instead: clarity beats cleverness in code someone will debug at 2 a.m. Two levels is fine, read left to right exactly like the loops it replaces: `[v for batch in batches for v in batch]`.

3.8 The `collections` module

Four specialised containers that delete whole categories of boilerplate.

`Counter`: frequency counting in one line

```
from collections import Counter

counts = Counter(["organic", "paid", "organic", "referral", "paid", "organic"])
counts.most_common(2)     # [('organic', 3), ('paid', 2)]
counts["unknown"]         # 0: a missing key never raises
counts.total()            # 6
```

FROM THE HISTORY OF DATA SCIENCE

Counting words to settle a 175-year-old argument

Of the 85 *Federalist Papers*, written in 1787–88 to persuade New Yorkers to ratify the US Constitution, twelve were claimed by both Alexander Hamilton and James Madison. In 1964 the statisticians Frederick Mosteller and David Wallace settled it by counting. Not grand ideas, but humble "function words": Hamilton wrote *upon* constantly, Madison hardly ever; Madison preferred *whilst* where Hamilton wrote *while*. Feeding those frequencies into a careful Bayesian analysis, they attributed all twelve disputed papers to Madison. It was one of the founding moments of statistical text analysis, and at its core is a `Counter`.

`defaultdict`: no more "if key not in dict"

```python
from collections import defaultdict

grouped = defaultdict(list)            # a missing key starts as []
for name, amount in transactions:
    grouped[name].append(amount)

totals = defaultdict(float)            # a missing key starts as 0.0
for name, amount in transactions:
    totals[name] += amount
```

`deque`: a queue with fast ends

`list.insert(0, x)` and `list.pop(0)` are O(n), because every remaining element has to shuffle along. A `deque` makes both ends O(1), and `maxlen` gives you a rolling window that forgets old items for free:

```python
from collections import deque

window = deque(maxlen=3)
for reading in [10, 20, 30, 40, 50]:
    window.append(reading)      # ends as deque([30, 40, 50])
moving_average = sum(window) / len(window)
```

`namedtuple`: records with names

```python
from collections import namedtuple

Measurement = namedtuple("Measurement", ["timestamp", "sensor_id", "value"])
reading = Measurement("2024-01-15T10:30", "S-001", 23.4)
reading.value              # 23.4 by name; reading[2] still works
reading._replace(value=24.0)   # a modified copy; the original is untouched
```

> **TIP**
> For anything beyond a trivial record, prefer `@dataclass`: it supports type hints, defaults, methods and mutability. Chapter 5 covers it.

3.9 Sorting

`sorted()` returns a new list; `list.sort()` sorts in place. Both take a `key` function that computes *what to sort by*:

```python
by_salary = sorted(employees, key=lambda e: e["salary"], reverse=True)

# Several fields: return a tuple. Negating a number flips its direction.
by_dept_then_salary = sorted(employees, key=lambda e: (e["dept"], -e["salary"]))

from operator import itemgetter
by_dept = sorted(employees, key=itemgetter("dept", "salary"))   # faster, clearer
```

Stability

Python's sort is **stable**: items that compare equal keep their original relative order. That's a guarantee, and it enables a neat technique: sort by the least important key first, then by the most important one, and the earlier ordering survives inside each group.

```
step_1 = sorted(employees, key=itemgetter("salary"), reverse=True)
step_2 = sorted(step_1, key=itemgetter("dept"))   # salary order kept per dept
```

> NOTE
> Python's sorting algorithm is called **Timsort**, written by Tim Peters for Python in 2002. It hunts for already-sorted runs in your data, which real data is full of, and it's stable by design. It was good enough that Java and Android later adopted it too.

3.10 Unpacking and star expressions

```
first, second, third = [1, 2, 3]
head, *rest = [1, 2, 3, 4, 5]          # head=1, rest=[2, 3, 4, 5]
first, *middle, last = [1, 2, 3, 4, 5]
a, b = b, a                            # swap, no temporary variable

describe(**{"name": "Ana", "dept": "data", "salary": 95_000})   # dict → keywords
describe(*["Bruno", "eng", 105_000])                            # list → positions
```

3.11 Try it yourself

Exercises 3.1 to 3.5 in notebook 03; solutions in notebook 09.

1. **Pick the structure.** Name the best collection, with a one-line justification, for: 50 million already-processed user IDs; the last 100 sensor readings; country code → country name; an RGB colour that must never change; word counts across 10,000 reviews.
2. **Fix the slow function.** `[i for i in all_ids if i not in known_ids]` takes minutes with 100,000 IDs on each side. Make it run in well under a second, and prove it with `%timeit`.
3. **Comprehension practice.** From a list of sales dicts, produce in single comprehensions: a list of revenues, a dict product → revenue, and the set of products with revenue above 100.
4. **Group without pandas.** Using `defaultdict`, write `group_by_key(records, key)`. This is `pandas.groupby` in eight lines, and writing it once makes Module 03 much clearer.
5. **Multi-key sorting.** Sort `employees` by department ascending, salary descending, name ascending, in a **single** `sorted()` call.

KEY TAKEAWAYS

- **Choose containers by complexity.** `list` membership is O(n); `set` and `dict` are O(1).
- The habit that matters: `set()` **anything you'll repeatedly test membership against.**
- Mutating list methods return `None`. Use `sorted()` when you want a new list.
- Tuples are records *and* the hashable sequence, so they can be dict keys.
- **Comprehensions** are the default way to build collections; keep them to two levels.
- `Counter`, `defaultdict`, `deque` and `namedtuple` delete whole categories of boilerplate.
- Python's sort is **stable**, which makes multi-pass sorting reliable.

CHAPTER 04

04

Functions That Behave

Signatures, scope, decorators and lazy generators

Notebook 04_functions_and_functional_programming.ipynb · About 90 minutes

IN THIS CHAPTER

- How to design a function signature that is hard to misuse.
- The mutable-default-argument trap, a bug that survives in production for years.
- Type hints and NumPy-style docstrings.
- Scope (the LEGB rule), `global`, `nonlocal` and closures.
- Decorators: how they work, and how to write useful ones.
- **Generators**: processing a file bigger than your RAM, one row at a time.
- `functools` and `itertools`: caching, partial application and lazy iteration.

A function is a promise. Give it these inputs and it will hand you back that output, and, ideally, it will do nothing else behind your back. This chapter is about writing functions that keep their promises, are hard to call wrongly, and stay fast and frugal when the data gets big.

4.1 Anatomy of a good function

Every function in this course's `src/` folder follows one shape. Each part earns its place:

```
def normalise(
    values: list[float],
    *,                          # everything after this is keyword-only
    method: str = "zscore",
    ddof: int = 1,
) -> list[float]:
    """Scale a sequence of numbers to a common range.

    Parameters
    ----------
    values : list of float
        The raw values. Must contain at least two elements.
    method : {"zscore", "minmax"}, default "zscore"
    ddof : int, default 1
        Delta degrees of freedom; 1 gives the sample standard deviation.
```

```
    Returns
    -------
    list of float

    Raises
    ------
    ValueError
        Fewer than two values, an unknown method, or zero spread.
    """
    if len(values) < 2:
        raise ValueError(f"need at least 2 values, got {len(values)}")
    ...
```

- **Type hints** (`list[float]`, `-> list[float]`) document intent and let tools catch mistakes.
- **A NumPy-style docstring**, the convention of the whole scientific Python world, explains parameters, return value, errors and examples.
- **Validation up front.** Fail early and loudly: a function that returns nonsense for bad input is far more expensive to debug than one that refuses it.
- **Helpful errors.** `f"unknown method {method!r}; expected 'zscore' or 'minmax'"` saves the caller a trip to the docs.

Why the lonely `*`?

Everything after a bare `*` is **keyword-only**. That turns a cryptic call into a self-documenting one, and it lets you reorder or add parameters later without breaking a single caller:

```
normalise(data, "minmax", 0)                # ✘ now a TypeError, and good riddance
normalise(data, method="minmax", ddof=0)    # ✔ obvious at a glance
```

> **TIP**
> A rule worth adopting: **every boolean or "mode" parameter should be keyword-only.** A call like `f(df, True, False)` is unreadable in a code review.

4.2 Parameters in full

Python's parameter system is unusually expressive. Here's the whole picture in one signature:

```
def demo(pos_only, /, standard, *args, kw_only, **kwargs):
    ...
demo(1, 2, 3, 4, kw_only=5, extra="x")
# pos_only=1  standard=2  args=(3, 4)  kw_only=5  kwargs={'extra': 'x'}
```

Everything before `/` is positional-only; everything after `*` is keyword-only. `*args` collects extra positional arguments into a tuple, `**kwargs` collects extra keywords into a dict. Their most common honest use is **forwarding** arguments to another function, for example a plotting wrapper that supplies house defaults the caller can still override:

```
def plot_with_defaults(x, y, **kwargs):
    settings = {"linewidth": 2, "marker": "o"} | kwargs   # caller wins
    ...
```

4.3 The mutable default argument trap

The most notorious gotcha in the language, and it follows directly from the object model in Chapter 2. **Default values are evaluated once, when the `def` line runs**, not each time you call the function. A mutable default is therefore shared by every call, forever.

```
def append_reading_broken(value, readings=[]):
    readings.append(value)
    return readings

append_reading_broken(1.0)   # [1.0]
append_reading_broken(2.0)   # [1.0, 2.0]   ← expected [2.0]
append_reading_broken(3.0)   # [1.0, 2.0, 3.0]
```



Figure 4.1 The default list is created once, when the function is defined, and every call that omits the argument shares and grows it.

The fix is to use `None` as a sentinel and build a fresh object inside:

```
def append_reading(value: float, readings: list[float] | None = None) -> list[float]:
    if readings is None:
        readings = []
    readings.append(value)
    return readings
```

> **WATCH OUT**
> The same trap applies to `{}`, `set()`, `pd.DataFrame()` and even `datetime.now()`, which gets frozen at definition time. The rule: **never put a mutable or time-dependent value in a default.**

4.4 Scope: the LEGB rule

When Python meets a name, it searches four namespaces in a fixed order and stops at the first match: Local, Enclosing, Global, Built-in.

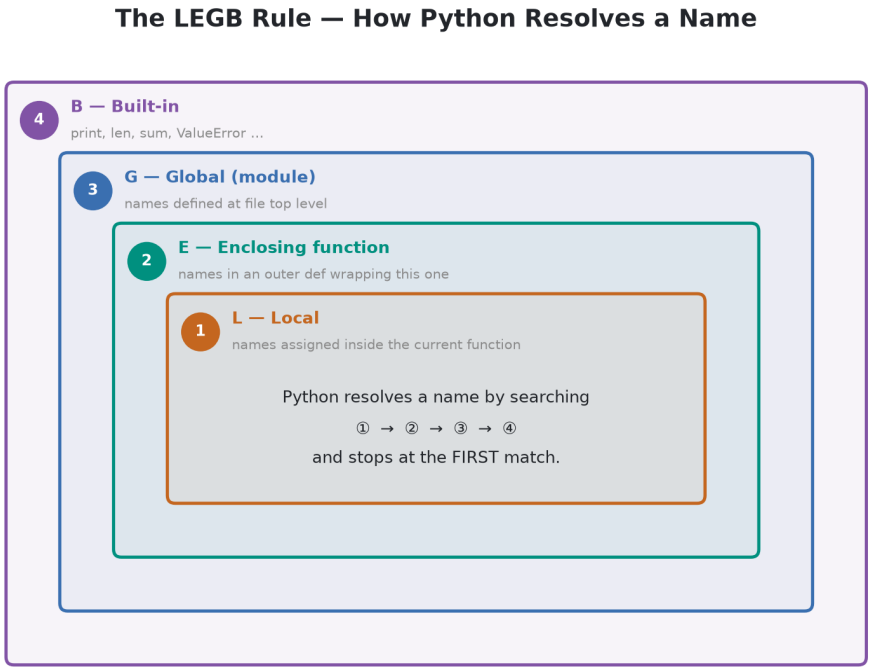


Figure 4.2 The LEGB rule: Python looks for a name from the innermost scope outwards.

Assignment makes a name local

One consequence trips everyone up once. If a function *assigns* to a name anywhere in its body, Python treats that name as local for the **whole** body:

```
counter = 0
def increment_broken():
    counter += 1      # UnboundLocalError: assignment made `counter` local
```

You *can* fix it with `global counter`, but a function that reads and writes module state is hard to test. The better fix is to take state in and hand new state back:

```
def increment(current: int) -> int:
    return current + 1
```

Closures: functions that remember

An inner function keeps a live link to the variables of the function that created it, even after that outer function has returned. That captured environment is called a **closure**:

```
def make_threshold_filter(threshold: float):
    def is_above(value: float) -> bool:
        return value > threshold        # remembered from the enclosing scope
    return is_above
```

```
above_100 = make_threshold_filter(100)
above_500 = make_threshold_filter(500)
[r for r in [50, 150, 600, 90, 1200] if above_100(r)]   # [150, 600, 1200]
```

To *rebind* a variable from the enclosing scope (not just read it), declare it `nonlocal`, the closure-sized cousin of `global`.

4.5 Decorators

A decorator is a function that takes a function and returns a replacement. The `@` syntax is pure shorthand: `@timed` above `def slow_sum` means exactly `slow_sum = timed(slow_sum)`.

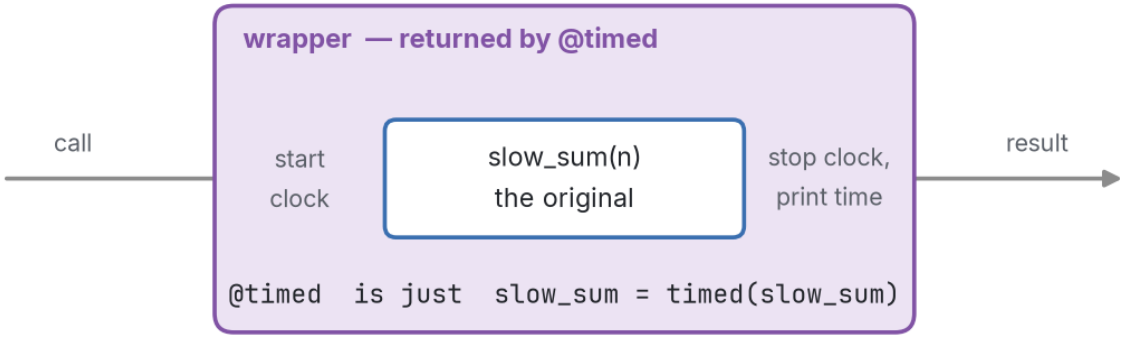


Figure 4.3 A decorator wraps the original function, adding behaviour before and after each call.

```
import functools, time

def timed(func):
    @functools.wraps(func)        # keep the original's name and docstring
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"[{func.__name__}] took {time.perf_counter() - start:.4f}s")
        return result
    return wrapper

@timed
def slow_sum(n: int) -> int:
    return sum(i**2 for i in range(n))
```

You'll meet decorators everywhere: `@property`, `@pytest.fixture`, `@app.get` in FastAPI (Module 22), `@task` in Airflow.

> **TIP**
> Always apply `functools.wraps`. Without it, debuggers, documentation tools and test reports all see a function called `wrapper`.

A decorator with arguments

That needs one more layer: a function that *returns* a decorator. A retry decorator is a realistic example, since flaky network calls to data APIs are a daily reality:

```python
def retry(times: int = 3, delay: float = 0.01):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as exc:
                    if attempt == times:
                        raise
                    time.sleep(delay)
        return wrapper
    return decorator

@retry(times=3)
def flaky_api_call(): ...
```

`lru_cache`: memoisation for free

`functools.lru_cache` remembers results by argument. For an expensive, deterministic function called repeatedly it's transformative. Naive recursive Fibonacci makes tens of millions of calls for `fibonacci(35)`; with the cache, the body runs just once for each of the 36 distinct values from 0 to 35.

```python
@functools.lru_cache(maxsize=128)
def fibonacci(n: int) -> int:
    return n if n < 2 else fibonacci(n - 1) + fibonacci(n - 2)
```

> **NOTE**
> The idea has a lovely name. In 1968 the British AI researcher Donald Michie described "memo functions" in the journal *Nature*: functions that keep a memo of answers they've already worked out. The word **memoisation** comes straight from his paper.

> **WATCH OUT**
> `lru_cache` needs **hashable** arguments, so it can't cache a function that takes a list or a DataFrame. It also keeps results in memory: set `maxsize` deliberately rather than `None` on a hot path.

4.6 Generators: the memory tool

A generator produces values **one at a time, on demand**, instead of building the whole collection up front. For data science this is the difference between processing a 50 GB file and getting a `MemoryError`. Any function containing `yield` is a generator function:

```
def read_records_lazy(n: int):
    for i in range(n):
        yield {"id": i, "value": i * 1.5}
```

A list of a million records holds a million records. A generator holds… one recipe, and a bookmark:

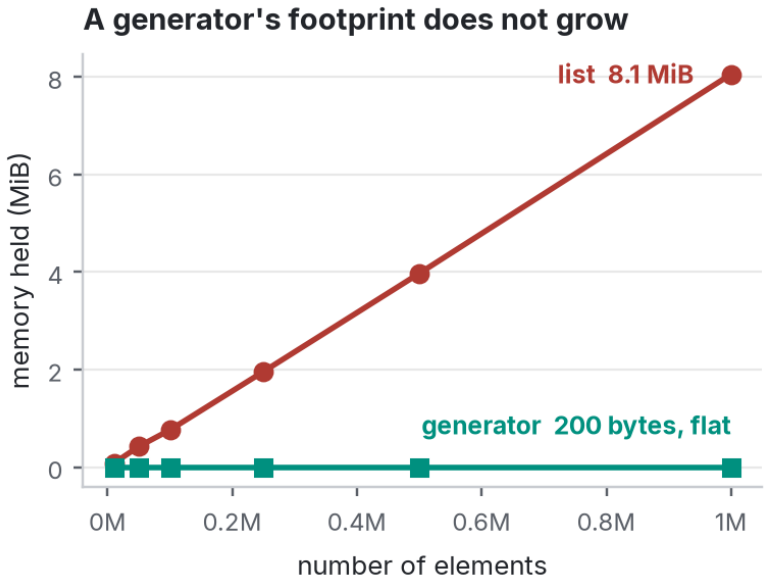


Figure 4.4 The list's memory grows with every element; the generator's stays at about two hundred bytes no matter how many items it will eventually produce.

Streaming a file

Chain generators together and you get a pipeline in which **nothing runs until the final loop asks for the next item.** A real file object iterates line by line in exactly the same way:

```
def parse_lines(lines):
    for line in lines:
        name, amount = line.strip().split(",")
        yield {"name": name, "amount": float(amount)}

def filter_large(records, minimum: float):
    for record in records:
        if record["amount"] >= minimum:
            yield record

pipeline = filter_large(parse_lines(open("big.csv")), minimum=100)
for record in pipeline:     # data starts flowing only now
    ...
```

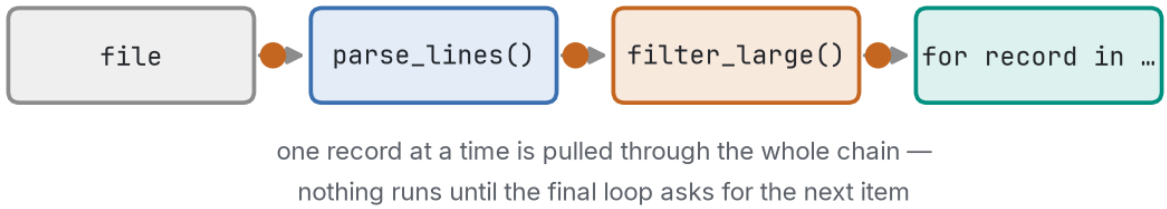


Figure 4.5 A lazy pipeline pulls one record at a time through every stage.

Generator expressions

Swap a list comprehension's square brackets for round ones and you get a generator expression. Prefer it whenever the result goes straight into a consumer like `sum`, `max`, `any` or `all`:

```
total = sum(v**2 for v in range(1_000_000))   # never builds the million-item list
any(v > 999_998 for v in values)              # stops at the first True
```

> **WATCH OUT**
>
> A generator is **single-use.** Once exhausted it yields nothing more, and it does so silently, without an error:
>
> ```
> gen = (x for x in [1, 2, 3])
> list(gen)    # [1, 2, 3]
> list(gen)    # []   ← empty, not an error
> ```

4.7 `map`, `filter`, `reduce` and `lambda`

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
list(map(lambda x: x**2, numbers))            # [1, 4, 9, 16, 25]
list(filter(lambda x: x % 2 == 0, numbers))   # [2, 4]
reduce(lambda acc, x: acc * x, numbers)       # 120
```

`lambda` creates a one-expression anonymous function. Use it only for short callables passed to something else; anything longer deserves a `def` and a name. In everyday Python a comprehension usually reads better than `map` and `filter`.

FROM THE HISTORY OF DATA SCIENCE

From a 1930s logic paper to Google's data centres

The keyword `lambda` comes from the **lambda calculus**, a formal system invented by the logician Alonzo Church in the 1930s to study what it means to compute something at all, years before electronic computers existed. Lisp borrowed it in the late 1950s, along with the habit of "mapping" a function over a list and "reducing" the results. In 2004 two Google engineers, Jeffrey Dean and Sanjay Ghemawat, published *MapReduce*, a way to spread exactly that pattern across thousands of machines. It powered a generation of big-data tools (Module 20). Not bad for a logic paper.

`functools.partial`: pre-filling arguments

```
def scale(value: float, *, factor: float, offset: float = 0.0) -> float:
    return value * factor + offset

to_fahrenheit = functools.partial(scale, factor=9 / 5, offset=32)
to_fahrenheit(20)     # 68.0
```

4.8 `itertools`: lazy iteration tools

```
import itertools

itertools.chain([1, 2], [3, 4])                    # 1, 2, 3, 4
itertools.islice(itertools.count(10), 5)           # 10, 11, 12, 13, 14
itertools.combinations(["a", "b", "c"], 2)         # ('a','b'), ('a','c'), ('b','c')
itertools.product([0.1, 0.5], ["l1", "l2"])        # every pairing: a grid search
itertools.groupby(sorted_records, key=lambda r: r["dept"])
```

WATCH OUT

`itertools.groupby` groups only **consecutive** equal items, so sort by the key first. Unsorted input gives you the same group several times.

TIP

`itertools.product` is exactly how a hyperparameter grid search enumerates its candidates. You'll see it again, wrapped inside `GridSearchCV`, in Module 13.

4.9 Pure functions and side effects

A **pure** function returns the same output for the same input and changes nothing outside itself. Pure functions are trivially testable, safe to run in parallel, and cacheable.

```
tax_rate = 0.2
def apply_tax_impure(order: dict) -> None:      # ✘ mutates input, reads a global
    order["total"] = order["amount"] * (1 + tax_rate)

def apply_tax(order: dict, rate: float) -> dict:  # ✔ everything in, everything out
    return {**order, "total": order["amount"] * (1 + rate)}
```

Not everything can be pure. Reading files, writing to databases and logging are side effects, and they're the whole point of the program. The professional pattern is to **push side effects to the edges**: a thin impure shell that does I/O, wrapped around a pure core that does the thinking. You'll build exactly that in this module's project.

4.10 Try it yourself

Exercises 4.1 to 4.5 in notebook 04; solutions in notebook 09.

1. **Fix the signature.** Rewrite `def summarise(data, True, 0.95, None)` as a well-designed signature with keyword-only options, defaults, type hints and a NumPy-style docstring.
2. **Spot the trap.** Call `add_feature(df_columns, new_cols=[])` three times with only its first argument. Explain the output, then fix it.
3. **Write a decorator.** `@validate_positive` should raise `ValueError` if any numeric positional argument is negative, and preserve the wrapped function's metadata.
4. **Generator pipeline.** Chain `parse(lines)`, `only_errors(records)` and `take(records, n)` over log lines, and confirm with `sys.getsizeof` that memory doesn't grow with input size.
5. **Closure counter.** Write `make_counter()` returning a function that yields 1, 2, 3… on successive calls, without `global`. Hint: `nonlocal`.

KEY TAKEAWAYS

- Make "mode" and boolean parameters **keyword-only** with `*`.
- **Never use a mutable default argument.** Use `None` as a sentinel.
- Type hints plus NumPy-style docstrings are the ecosystem's conventions.
- Names resolve by **LEGB**; assigning inside a function makes a name local.
- A **decorator** wraps a function; always use `functools.wraps`.
- `lru_cache` turns an expensive deterministic function into a lookup.
- **Generators trade time for memory**: constant footprint whatever the input size, and single-use.
- Prefer **pure functions**; push I/O and mutation to the edges.

CHAPTER 05

05

Objects With a Purpose

Classes, dataclasses, and the scikit-learn estimator contract

Notebook 05_object_oriented_programming.ipynb · About 75 minutes

IN THIS CHAPTER

- When a class is the right tool, and when a plain function is better.
- Classes, instances, and the trap hiding in class attributes.
- Dunder methods: making your objects work with `print`, `len`, `==` and `[]`.
- `@dataclass`, which deletes most of the boilerplate.
- `@property` for validated, computed attributes.
- Inheritance versus composition, and why the industry prefers composition.
- **The scikit-learn estimator pattern**, the interface you'll implement for real.

Object-oriented programming has a reputation for ceremony: diagrams, hierarchies, and classes that exist to hold other classes. In data science you need very little of that. You need a clear answer to one question (*when does a class actually earn its keep?*) and one pattern you'll use for the rest of your career.

5.1 When should a data scientist write a class?

Honest answer: **less often than tutorials suggest.** Most analysis code should be functions operating on DataFrames. Reach for a class when you have **state and behaviour that belong together**.

Situation	Use
Transform data once, return a result	a **function**
Group related constants	a **module** or an `Enum`
Carry a fixed set of fields around	a `@dataclass`
Something *learns* parameters, then *applies* them	a **class**
Several interchangeable implementations behind one interface	a **class hierarchy** or a `Protocol`

The fourth row is the big one. A scaler learns a mean and a standard deviation from training data, then applies them to new data. That *fitted state* is exactly what an object is for.

FROM THE HISTORY OF DATA SCIENCE

Classes were invented to simulate ships and queues

The first language with classes and objects was **Simula 67**, built in Oslo by Ole-Johan Dahl and Kristen Nygaard at the Norwegian Computing Center. They weren't trying to invent a programming philosophy; they were writing *simulations* (of ships, harbours, queues of customers) and wanted each simulated thing to bundle its own data with its own behaviour. Alan Kay later coined the term "object-oriented", and Dahl and Nygaard received the Turing Award in 2001. The original motive is still the right one: **a class is for something with state that does things.**

5.2 Classes and instances

```
class Dataset:
    """A named collection of numeric observations."""

    version = "1.0"                      # CLASS attribute: shared by all instances

    def __init__(self, name: str, values: list[float]) -> None:
        self.name = name                 # INSTANCE attributes: one per object
        self.values = list(values)       # a copy, so the caller's list stays independent

    def mean(self) -> float:
        return sum(self.values) / len(self.values)

sales = Dataset("q1_sales", [120.0, 340.5, 89.0, 560.25, 210.0])
sales.mean()
```

Two things worth knowing. `__init__` is not a constructor: the object already exists when it runs, and its job is to set up instance attributes. And `self` is just the instance the method was called on: `sales.mean()` is really `Dataset.mean(sales)`.

The class-attribute trap

Class attributes are shared. If a **mutable** one is changed through any instance, every instance sees the change. It's Chapter 2's aliasing lesson in a new costume:

```
class BrokenLogger:
    entries = []                         # BUG: one list for every instance, ever

    def log(self, message: str) -> None:
        self.entries.append(message)

a, b = BrokenLogger(), BrokenLogger()
a.log("from a"); b.log("from b")
b.entries        # ['from a', 'from b']   ← the same list!
```

The fix is to create per-instance state inside `__init__`: `self.entries: list[str] = []`.

5.3 Dunder methods

"Dunder" (double underscore) methods let your objects join in with Python's built-in syntax. Implement a handful and a class feels native:

```
class Measurement:
    def __init__(self, value: float, unit: str) -> None:
        self.value, self.unit = value, unit

    def __repr__(self) -> str:      # for developers: looks like code that rebuilds it
        return f"Measurement(value={self.value!r}, unit={self.unit!r})"

    def __str__(self) -> str:       # for people: used by print()
        return f"{self.value:g} {self.unit}"

    def __eq__(self, other) -> bool:          # value equality instead of identity
        if not isinstance(other, Measurement):
            return NotImplemented
        return (self.value, self.unit) == (other.value, other.unit)

    def __hash__(self) -> int:      # defining __eq__ removes the default hash
        return hash((self.value, self.unit))

    def __lt__(self, other) -> bool:          # enables <, and therefore sorted()
        return self.value < other.value

    def __add__(self, other) -> "Measurement":  # enables +
        return Measurement(self.value + other.value, self.unit)
```

> **TIP**
> Implement `__repr__` on every class you write. Containers, debuggers and error messages all show it, and your future self will be reading it at some inconvenient hour.

Container dunders

Add `__len__`, `__getitem__` and `__iter__` and your class works with `len()`, `[]`, `for`, `in`, `max()`, `sum()` and unpacking, all for free. Supporting slices in `__getitem__` costs one `isinstance(index, slice)` check and makes the class behave the way users expect a sequence to.

5.4 `@dataclass`: delete the boilerplate

Most classes in data work just hold fields. `@dataclass` writes `__init__`, `__repr__` and `__eq__` for you from type-annotated attributes:

```
from dataclasses import dataclass, field

@dataclass
class ExperimentConfig:
    name: str
    learning_rate: float = 0.01
    max_epochs: int = 100
    features: list[str] = field(default_factory=list)   # a fresh list per instance
    seed: int = 42
```

Notice `field(default_factory=list)`. The dataclass machinery *refuses* a plain `= []` default, catching Chapter 4's mutable-default trap at class-definition time instead of letting it bite at runtime.

Frozen dataclasses for configuration

`frozen=True` makes instances immutable and hashable. It's the right choice for configuration: no code path can quietly rewrite a hyperparameter halfway through a run. `slots=True` stores attributes in a fixed array instead of a per-instance dict, saving real memory when you create millions of objects. And `__post_init__` is the place for validation:

```
@dataclass(frozen=True, slots=True)
class ModelParams:
    alpha: float
    l1_ratio: float = 0.5

    def __post_init__(self) -> None:
        if not 0 <= self.l1_ratio <= 1:
            raise ValueError(f"l1_ratio must be in [0, 1], got {self.l1_ratio}")

params = ModelParams(alpha=0.1)
params.alpha = 0.5        # FrozenInstanceError
```

5.5 `@property`: computed and validated attributes

A property looks like an attribute from outside but runs code inside. Use it to **validate on assignment**, or to expose a **derived value** that can never drift out of sync with its inputs:

```
class Portfolio:
    def __init__(self, weights: dict[str, float]) -> None:
        self.weights = weights                    # runs the setter below

    @property
    def weights(self) -> dict[str, float]:
        return self._weights

    @weights.setter
    def weights(self, value: dict[str, float]) -> None:
        total = sum(value.values())
        if not np.isclose(total, 1.0):            # float-safe, per Chapter 2
            raise ValueError(f"weights must sum to 1.0, got {total:.4f}")
        self._weights = dict(value)

    @property
    def n_assets(self) -> int:                    # read-only and always in sync
        return len(self._weights)
```

5.6 Inheritance versus composition

Inheritance expresses "**is a**". Composition expresses "**has a**". Modern practice strongly favours composition, because deep inheritance trees become impossible to reason about.

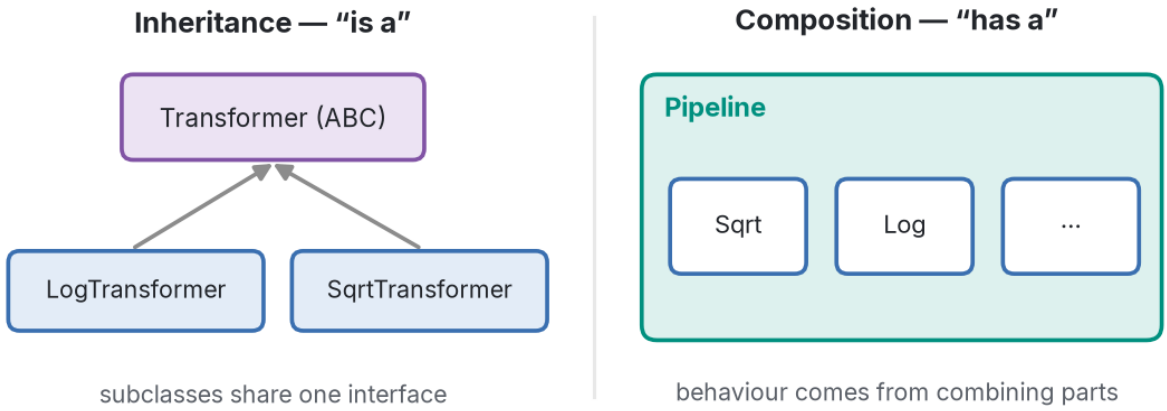


Figure 5.1 Inheritance shares an interface down a tree. Composition builds behaviour by combining independent parts.

Inheritance still has a good use: defining a shared **interface**. An abstract base class turns "every transformer must have a `transform` method" from a polite convention into a rule the interpreter enforces:

```
from abc import ABC, abstractmethod

class Transformer(ABC):
    @abstractmethod
    def transform(self, values: np.ndarray) -> np.ndarray: ...

class LogTransformer(Transformer):
    def transform(self, values):
        return np.log1p(values)        # log(1 + x): accurate near 0, tolerates zeros

Transformer()     # TypeError: can't instantiate an abstract class
```

Composition is the pattern you'll use more. A `Pipeline` doesn't *inherit* from `Transformer`; it *holds a list of them* and calls each in turn, which is exactly how scikit-learn's own `Pipeline` works:

```
class Pipeline:
    def __init__(self, steps: list[Transformer]) -> None:
        self.steps = steps

    def transform(self, values):
        for step in self.steps:
            values = step.transform(values)
        return values
```

> **NOTE**
> "Favour object composition over class inheritance" is one of the central pieces of advice in *Design Patterns* (1994), the so-called "Gang of Four" book by Gamma, Helm, Johnson and Vlissides. Thirty years on, it still holds.

5.7 The scikit-learn estimator pattern

Every scikit-learn object follows one contract. Implement it and your own code drops straight into a `Pipeline`, a `GridSearchCV` or `cross_val_score`. It's the single most valuable class-writing skill in applied data science.

Method	Contract
`__init__(...)`	**Only store** the arguments, unchanged, under the same names. No validation, no computation.
`fit(X, y=None)`	**Learn** from training data. Store results with a trailing underscore (`self.mean_`). Return `self`.
`transform(X)`	**Apply** what was learned. Never learn here.
`fit_transform(X)`	Free, inherited from `TransformerMixin`.

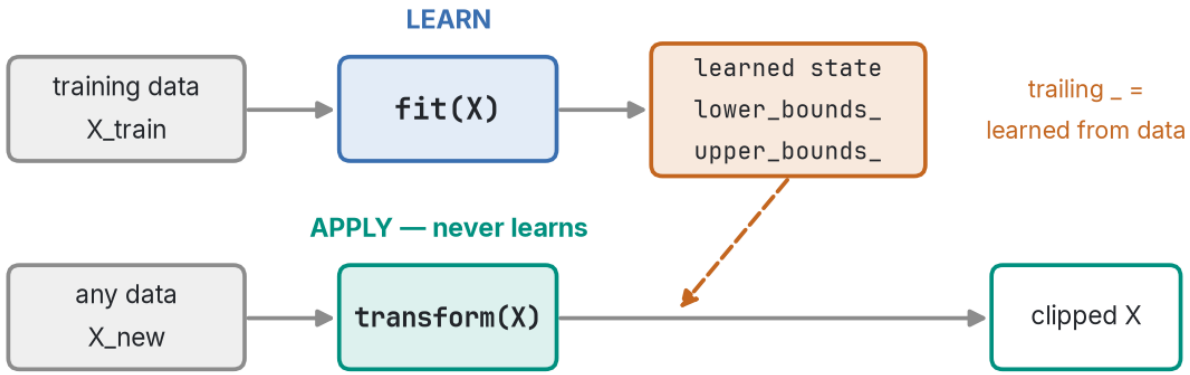


Figure 5.2 The estimator contract: fit learns state from training data; transform applies that state to any data and never learns.

> **WATCH OUT**
> The trailing underscore isn't decoration: it marks an attribute as *learned from data*. `self.lower_percentile` is a hyperparameter the user chose; `self.lower_bounds_` is something `fit` computed. Mixing them up breaks cloning, grid search and your colleagues' expectations.

Here's a genuinely useful custom transformer: it learns per-column percentile bounds from the training data and clips extreme values to them, taming outliers without dropping rows.

```
from sklearn.base import BaseEstimator, TransformerMixin

class OutlierClipper(BaseEstimator, TransformerMixin):
    def __init__(self, lower_percentile: float = 1.0,
                 upper_percentile: float = 99.0) -> None:
        self.lower_percentile = lower_percentile      # just store them
        self.upper_percentile = upper_percentile

    def fit(self, X, y=None):
        X = np.asarray(X, dtype=float)
        self.lower_bounds_ = np.percentile(X, self.lower_percentile, axis=0)
```

```
        self.upper_bounds_ = np.percentile(X, self.upper_percentile, axis=0)
        self.n_features_in_ = X.shape[1]
        return self                          # enables .fit(X).transform(X)

    def transform(self, X):
        if not hasattr(self, "lower_bounds_"):
            raise RuntimeError("call fit() before transform()")
        return np.clip(np.asarray(X, dtype=float),
                       self.lower_bounds_, self.upper_bounds_)
```

The payoff: it composes

Because it honours the contract, our class sits alongside scikit-learn's own steps as an equal:

```
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline as SklearnPipeline
from sklearn.preprocessing import StandardScaler

model = SklearnPipeline([
    ("clip", OutlierClipper(lower_percentile=2, upper_percentile=98)),
    ("scale", StandardScaler()),
    ("ridge", Ridge(alpha=1.0)),
])
model.fit(X_train, y_train)
[k for k in model.get_params() if k.startswith("clip__")]
# ['clip__lower_percentile', 'clip__upper_percentile']  ← tunable by GridSearchCV
```

`get_params` works only because `__init__` did nothing but store its arguments. That's what lets `GridSearchCV` tune our custom step in Module 13.

FROM THE HISTORY OF DATA SCIENCE

A summer project that became the industry's grammar

scikit-learn began in 2007 as a Google Summer of Code project by David Cournapeau. A team at INRIA, France's national computing research institute (Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort and Vincent Michel), picked it up and made the first public release in 2010. In 2013 its developers wrote a paper about their API design, arguing that a small, consistent interface (`fit`, `predict`, `transform`, plain parameters in `__init__`) matters more than any single algorithm. They were right: today dozens of other libraries copy that interface so that they can plug into scikit-learn's pipelines.

5.8 Try it yourself

Exercises 5.1 to 5.5 in notebook 05; solutions in notebook 09.

1. **Class or function?** Decide, with one sentence each: Celsius → Fahrenheit; database connection settings; a tokeniser that builds a vocabulary from a corpus and then encodes new text; the median of a list.
2. **Dunder practice.** Give `Measurement` a `__sub__`, a `__mul__` by a plain number, and a `__format__` so that `f"{m:.2f}"` works.
3. **Dataclass conversion.** Rewrite `Dataset` as a `@dataclass`, keeping `mean()` and `summary()`. How many lines disappeared?
4. **Build an estimator.** `RareCategoryGrouper` learns in `fit` which categories appear fewer than `min_frequency` times, and in `transform` replaces them with `"__other__"`. Check it inside a scikit-learn `Pipeline`.
5. **Find the bug.** A `Model` class keeps `history = []` at class level and appends to it in `fit`. Fit two instances once each and explain `len(m1.history)`.

KEY TAKEAWAYS

- Use a class when **state and behaviour belong together**, most often when something is *fitted* and then *applied*.
- **Class attributes are shared.** Initialise per-instance state in `__init__`.
- Implement `__repr__` on every class you write.
- `@dataclass` removes boilerplate; `frozen=True` gives immutable, hashable configuration.
- `@property` validates on assignment and exposes derived values safely.
- **Prefer composition to inheritance**; use abstract base classes to define interfaces.
- Follow the **scikit-learn contract**: `__init__` stores, `fit` learns into `attribute_` and returns `self`, `transform` applies. Then your code composes with the whole ecosystem.

CHAPTER 06

06
Files Without Fear

Paths, context managers, and choosing a data format on evidence

Notebook 06_files_and_data_formats.ipynb · About 75 minutes

IN THIS CHAPTER

- `pathlib`, the modern way to handle paths, and why string concatenation breaks.
- Context managers (`with`), and how to write your own.
- CSV: the format everyone uses and nobody reads the spec for.
- JSON: nested data, and the types it silently can't represent.
- **Parquet**: columnar storage, and why it's smaller and faster than CSV.
- A measured comparison, so you can choose a format on evidence.

Data arrives as files, and files are where a surprising number of bugs are born: a path that only works on one laptop, a write that dies halfway, a date that quietly became a string. None of this is glamorous. All of it decides whether your numbers are right.

6.1 `pathlib`: stop building paths with strings

Gluing paths together with `+` is fragile. It breaks across operating systems, doubled slashes creep in, and nothing checks your work. `pathlib.Path` treats a path as a structured object, and the `/` operator joins pieces with the right separator for whatever OS you're on:

```
from pathlib import Path

REPO_ROOT = find_repo_root()          # see below
RAW_DIR = REPO_ROOT / "data" / "raw"
RAW_DIR.mkdir(parents=True, exist_ok=True)   # safe to run again and again
```

`parents=True` creates missing intermediate folders; `exist_ok=True` makes the call idempotent, so re-running the notebook is harmless.

Find the project root, don't hard-code it

Counting `.parent` calls breaks the moment a notebook moves one level. Searching upwards for a file that only exists at the root doesn't:

```python
def find_repo_root(marker: str = "pyproject.toml") -> Path:
    for candidate in [Path.cwd(), *Path.cwd().parents]:
        if (candidate / marker).exists():
            return candidate
    raise FileNotFoundError(f"no {marker} found above {Path.cwd()}")
```

Paths come apart into named pieces

```python
sample = RAW_DIR / "customers_2024_q1.csv"
sample.name                      # 'customers_2024_q1.csv'
sample.stem                      # 'customers_2024_q1'
sample.suffix                    # '.csv'
sample.parent                    # .../data/raw
sample.with_suffix(".parquet")   # same place, new extension
sorted(RAW_DIR.glob("*.csv"))    # one level; .rglob() recurses
```

> **TIP**
> Define your paths **once**, near the top of a notebook, relative to a project root you compute. An absolute path like `/Users/ana/Desktop/project/data.csv` is the single most common reason a colleague can't run your notebook.

6.2 Context managers

A file must be closed. Leaving files open leaks operating-system handles and, worse for data work, buffered writes may never reach the disk. The `with` statement guarantees cleanup **even if an exception is raised** inside the block:

```python
with open(path, "a", encoding="utf-8") as f:
    f.write("second line\n")
# f is closed here, error or no error

path.write_text("overwritten\n", encoding="utf-8")   # pathlib shortcut
path.read_text(encoding="utf-8")
```

Writing your own: an atomic write

Any setup-and-teardown pair is a candidate for a context manager. `@contextmanager` turns a generator into one: everything before `yield` is setup; everything after is guaranteed teardown.

Here's a genuinely useful one. Scheduled data jobs crash, and a half-written output file that replaces a good one silently poisons everything downstream. The fix: write to a temporary file, and swap it into place **only on success**:

```python
from contextlib import contextmanager

@contextmanager
def atomic_write(path: Path, encoding: str = "utf-8"):
    temp_path = path.with_suffix(path.suffix + ".tmp")
    handle = temp_path.open("w", encoding=encoding)
    try:
        yield handle                        # the body of the `with` runs here
    except Exception:
```

```
        handle.close()
        temp_path.unlink(missing_ok=True)  # throw the partial file away
        raise                              # and let the caller see the error
    else:
        handle.close()
        temp_path.replace(path)            # atomic on POSIX: old or new, never half
```

Crash inside the `with` and the original file is untouched, with no `.tmp` left behind.

6.3 A dataset worth measuring

To compare formats meaningfully, the notebook builds 200,000 synthetic orders with a realistic mix of columns: integer IDs, dates, low-cardinality strings (`channel`, `country`), money amounts, quantities and a boolean.

> TIP
> Always check `df.dtypes` right after building or loading a DataFrame. A numeric column that arrived as `object` is the most common cause of a slow, wrong analysis, and it's invisible in `.head()`.

6.4 CSV: universal, and full of traps

CSV is a *convention*, not a strict standard. Everybody supports it, which is its only real advantage.

> FROM THE HISTORY OF DATA SCIENCE
> **A format nobody wrote down for thirty years**
> Comma-separated values were in use on mainframes by the early 1970s. Yet the first serious attempt to document the format, RFC 4180, only appeared in **2005**, and even that is an "informational" memo describing common practice, not a binding standard. That's why every tool has its own opinions about quoting, line endings, separators and decimal marks, and why "it's just a CSV" is famous last words.

Write with `index=False`. Otherwise pandas writes the row index as an unnamed first column, which comes back as `Unnamed: 0` the next time anyone reads the file.

What CSV loses

A CSV file is *just text*. Every type annotation is thrown away on write, and pandas has to guess them all back on read. Round-trip the orders table and `order_date` comes back as a plain string, so date arithmetic fails or, worse, text sorting puts `2024-10-01` before `2024-2-01`.

FROM THE HISTORY OF DATA SCIENCE

When spreadsheets renamed human genes

Type-guessing isn't just a pandas problem. Excel, helpfully, converts text that *looks* like a date into a date. Gene symbols such as SEPT2 (Septin 2) and MARCH1 became "2-Sep" and "1-Mar". In 2016 a study by Ziemann, Eren and El-Osta found the problem in roughly one in five genomics papers that shipped gene lists as Excel supplements. The fix eventually came from the other side: in 2020 the HUGO Gene Nomenclature Committee **renamed** the affected genes (SEPT1 became SEPTIN1, MARCH1 became MARCHF1) so that spreadsheets would leave them alone. When software guesses your types, sooner or later it guesses wrong.

The fix is to **tell** pandas what you expect instead of letting it guess:

```
reloaded = pd.read_csv(
    csv_path,
    parse_dates=["order_date"],
    dtype={
        "order_id": "int32",
        "customer_id": "int32",
        "channel": "category",     # repeated strings → big memory savings
        "country": "category",
        "quantity": "int8",
    },
)
```

An explicit dtype map is both faster and safer than inference, and `category` cuts memory dramatically for columns with few distinct values.

The other CSV traps

Trap	Symptom	Fix
Wrong encoding	`UnicodeDecodeError`	`encoding="latin-1"` or `"cp1252"`
European decimals	`1.234,56` read as text	`decimal=",", sep=";"`
Leading zeros	`00123` becomes `123`	`dtype={"zip": str}`
Huge numeric IDs	precision lost above 2^{53}	`dtype={"id": str}`
`"NA"`, `"N/A"`, `"-"`	not recognised as missing	`na_values=["NA", "-"]`
File bigger than RAM	`MemoryError`	`chunksize=100_000`

That last row is Chapter 4's generator idea in CSV form: read a chunk, process it, let it go.

```
total = 0.0
for chunk in pd.read_csv(csv_path, chunksize=50_000, usecols=["amount"]):
    total += chunk["amount"].sum()
```

6.5 JSON: nested data

JSON handles hierarchy, which CSV can't, and it's the format of almost every web API you'll pull data from. Write it with `indent=2` (readable, diffable) and `ensure_ascii=False` (accented characters stay as themselves).

Flattening for analysis

`pd.json_normalize` turns nested structures into a flat table: the bridge between an API response and a DataFrame.

```
flat = pd.json_normalize(
    api_response,
    record_path="results",                    # the list that becomes rows
    meta=["request_id", ["meta", "page"]],    # fields copied onto every row
)
```

WATCH OUT

JSON has no date type and no NaN. Dates become strings, and `NaN` and `Infinity` aren't valid JSON at all. Python's `json` module writes them anyway, producing files that stricter parsers in other languages reject. Pass `allow_nan=False` to catch it, and write `null` deliberately instead.

NOTE

JSON was popularised in the early 2000s by Douglas Crockford, who liked to say he *discovered* it rather than invented it: it's simply the syntax JavaScript already used for literal objects. It became formal standards later (ECMA-404 in 2013).

6.6 Parquet: the format for data science

Parquet stores data **column by column** instead of row by row, with per-column compression and the schema embedded in the file.

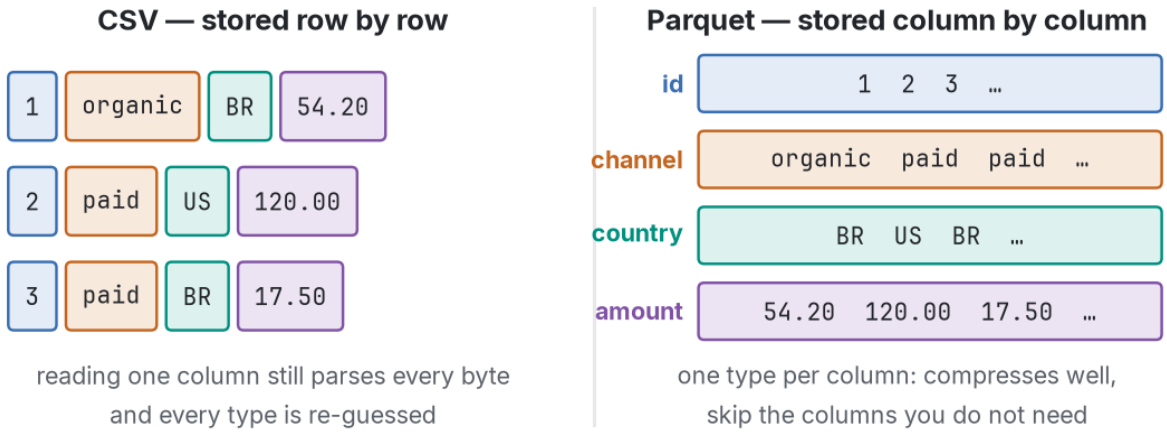


Figure 6.1 Row storage keeps each record together. Column storage keeps each field together, all of one type.

Three consequences follow directly from that layout:

1. **Compression is far better.** A column holds one type, often with few distinct values.
2. **Reading a subset of columns is nearly free.** The other columns are never touched. This is called *column pruning*.
3. **The schema travels with the data.** Dtypes survive the round trip exactly: `orders.equals(pd.read_parquet(path))` is `True`.

```
orders.to_parquet(path, engine="pyarrow", compression="snappy", index=False)
pd.read_parquet(path, columns=["channel", "amount"])    # reads only two columns
```

FROM THE HISTORY OF DATA SCIENCE

From Google's Dremel to your laptop

In 2010 Google published a paper on **Dremel**, the system it used to run interactive queries over trillions of rows. One of its key ideas was storing even nested records in a columnar layout. Engineers at Twitter and Cloudera took that idea and in 2013 released **Apache Parquet** as an open format. The trick that lets a data warehouse scan only the columns a query needs is the same one that makes your `read_parquet(columns=[...])` fast.

6.7 A measured comparison

Enough assertion. Here's the same 200,000-row table in three formats, measured on the machine that built this book:

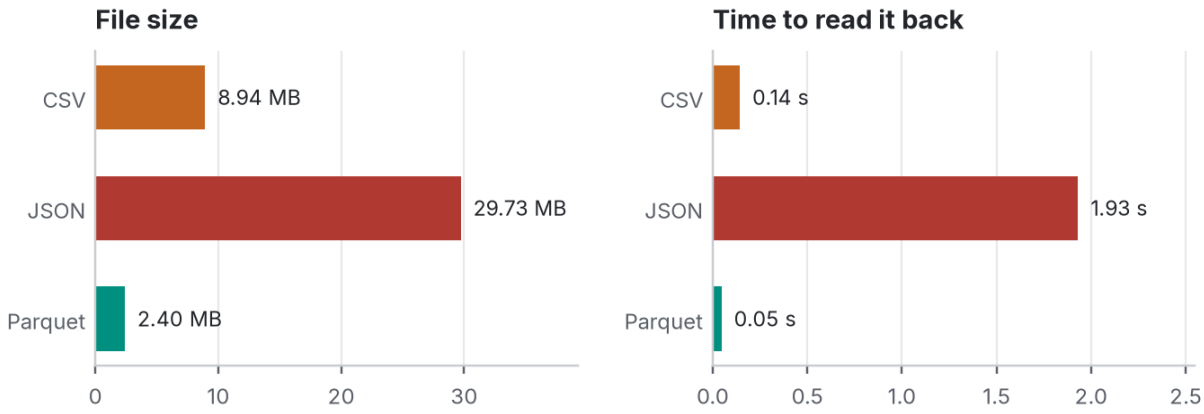


Figure 6.2 Size on disk and time to read back the same table. Your exact numbers will differ; the ranking won't.

6.8 Choosing a format

Format	Use it when	Avoid it when
CSV	A human must open it; exchanging with a non-technical team	Data is large, typed or nested
JSON	Nested data; API payloads; configuration	Tabular data at any real scale
Parquet	**Any analytical dataset you control:** the default	Someone needs to read it in a text editor
Pickle	Short-lived, Python-only caching	*Anything shared or long-lived*
Feather	Very fast temporary hand-off between pandas/R sessions	Long-term archives

> WATCH OUT
> **Never load a pickle from an untrusted source.** Unpickling can execute arbitrary code by design, so a malicious `.pkl` file is simply a program you agreed to run. Pickles are also fragile across library versions.

The convention this course follows

```
data/raw/         ← exactly as received, never modified (CSV, JSON, whatever)
data/processed/   ← cleaned, typed, analysis-ready → always Parquet
```

A small method chain shows the convention in action: read raw, fix types, derive a column, filter, and write processed Parquet.

```
clean = (
    pd.read_csv(csv_path, parse_dates=["order_date"])
    .assign(
        channel=lambda df: df["channel"].astype("category"),
        revenue=lambda df: (df["amount"] * df["quantity"]).round(2),
    )
    .loc[lambda df: df["amount"] > 0]
)
clean.to_parquet(PROCESSED_DIR / "orders_clean.parquet", index=False)
```

6.9 Try it yourself

Exercises 6.1 to 6.5 in notebook 06; solutions in notebook 09.

1. **Path manipulation.** From `Path("data/raw/sales_2024_q1.csv")`, using only `pathlib`, produce `data/processed/sales_2024_q1.parquet`, the year and quarter as strings, and a sibling `sales_2024_q1_backup.csv`.
2. **Diagnose a broken CSV.** Read a semicolon-separated file with `001;Café;1.234,56;15/03/2024` rows so that `id` keeps its leading zeros, `price` is a float and `date` is a datetime.
3. **Nested JSON.** Flatten an order with a nested `customer` and a list of `items` into one row per item, carrying the order fields onto each row.

4. **Prove column pruning.** Build a 50-column, 500,000-row DataFrame. Time reading **one** column from Parquet and from CSV, and explain the ratio using the storage layout.
5. **Safe writing.** Extend `atomic_write` with `backup: bool`, keeping the previous version as `<name>.bak`.

KEY TAKEAWAYS

- Use `pathlib`, never string concatenation. Compute a project root; never hard-code absolute paths.
- **Always use** `with` for files: it closes them even when an exception fires.
- **CSV loses every dtype.** Pass `parse_dates` and an explicit `dtype` map; write with `index=False`.
- **JSON has no date and no NaN.** Convert deliberately before writing.
- **Parquet is the default for analytical data**: smaller, faster, self-describing, and it supports column pruning.
- **Never unpickle untrusted data.**
- Convention: `data/raw/` is immutable input; `data/processed/` is Parquet output.

CHAPTER 07

07
Failing Loudly

Exceptions, logging and tests for pipelines you can trust

Notebook 07_errors_logging_and_testing.ipynb · About 90 minutes

IN THIS CHAPTER

- Exceptions: catching the right one, and why `except Exception:` hides bugs.
- `try / except / else / finally`, and exception chaining with `raise ... from`.
- Custom exceptions that make a pipeline's failures explain themselves.
- **Logging**, and why `print` doesn't survive contact with production.
- Defensive validation: fail fast, fail loudly.
- `pytest`: testing data transformations, including the tests that catch bugs unit tests usually miss.

There are two kinds of broken programs. One crashes, tells you where, and gets fixed by lunchtime. The other keeps running and produces a number that looks perfectly reasonable and is wrong. This chapter is about turning the second kind into the first. It's the difference between a script that *seems* to work and a pipeline you'd let run unattended.

7.1 The exception hierarchy

Exceptions form a family tree of classes. Catching a class catches all of its descendants, which is exactly why catching too high up the tree is dangerous.

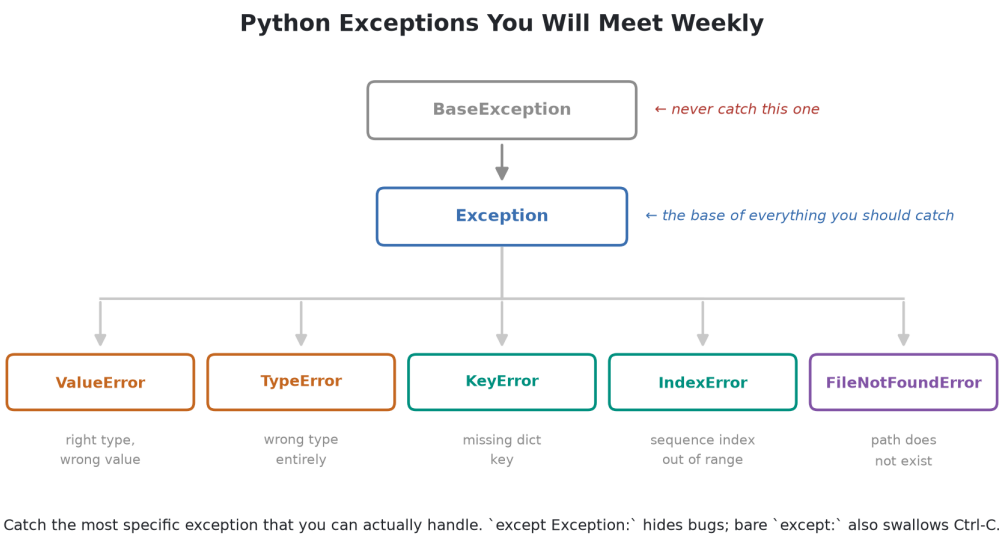


Figure 7.1 The exceptions you'll meet weekly, and how they're related.

You can verify the relationships yourself: `issubclass(KeyError, LookupError)` and `issubclass(FileNotFoundError, OSError)` are both `True`. One relationship is deliberately absent. `KeyboardInterrupt` and `SystemExit` inherit from `BaseException`, **not** `Exception`, so that `except Exception:` won't swallow your Ctrl-C. A bare `except:` catches `BaseException` too, which makes a runaway job impossible to stop.

> FROM THE HISTORY OF DATA SCIENCE
>
> ### The first actual bug
>
> On 9 September 1947, operators of the Harvard Mark II computer traced a fault to a moth trapped in one of its relays. They taped it into the logbook with the note "First actual case of bug being found". Grace Hopper, who worked on the machine and loved telling the story, helped make it famous. Engineers had been calling glitches "bugs" since at least Thomas Edison's day, which is the joke: this time it was a *literal* one. The page, moth included, is now held by the Smithsonian.

7.2 Catch narrowly

The rule: **catch the most specific exception you can actually do something about.** Let everything else propagate.

```
def parse_amount_bad(raw: str) -> float:
    try:
        return float(raw)
    except Exception:
        return 0.0          # a typo and a null now look identical

def parse_amount_good(raw: str | None) -> float | None:
    if raw is None or raw.strip() == "":
        return None         # genuinely missing: an expected, legitimate state
    try:
        return float(raw)
    except ValueError:
```

```
        raise ValueError(f"could not parse {raw!r} as a number") from None
```

> **WATCH OUT**
> `parse_amount_bad` turns `"N/A"` into `0.0`. Sum a column of those and you get a revenue figure that is confidently, silently wrong. **Returning a plausible default on failure is how bad numbers reach dashboards.**

7.3 `try / except / else / finally`

All four clauses have distinct jobs:

Clause	Runs when	Use it for
`try`	always	the operation that might fail
`except`	a matching exception was raised	handling that specific failure
`else`	**no** exception was raised	follow-on work, kept out of `try`
`finally`	**always**, even after `return` or a re-raise	cleanup

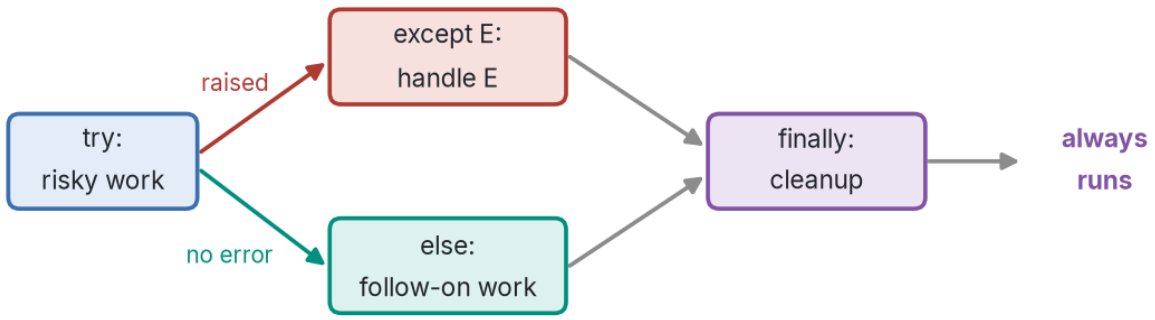


Figure 7.2 The four clauses as a flow: finally runs on every path.

```
def load_dataset(path: Path) -> pd.DataFrame | None:
    try:
        df = pd.read_csv(path)
    except FileNotFoundError:
        return None
    except pd.errors.EmptyDataError:
        return None
    else:
        print(f"loaded {len(df)} rows")    # outside `try`: its errors aren't masked
        return df
    finally:
        print("cleanup would go here")     # runs even after the returns above
```

Why the `else`? Code there runs only on success, and because it's outside the `try`, you can't accidentally catch an exception raised by the follow-on work.

Exception chaining: `raise ... from`

When you translate a low-level error into a domain-level one, keep the original. `from exc` records the cause, so the traceback shows both the message you wrote *and* which file actually went missing:

```
def load_config(path: Path) -> dict:
    try:
        return json.loads(path.read_text())
    except FileNotFoundError as exc:
        raise RuntimeError(f"configuration required at {path}") from exc
```

7.4 Custom exceptions

Your own exception types let callers handle *your* failures specifically. In a pipeline, that's how you tell "the data is bad" from "the code is broken", two problems with very different owners. Give them a shared base class so that one `except` can catch them all:

```
class DataValidationError(Exception):
    """Base class for every data-quality failure in this project."""

class MissingColumnsError(DataValidationError):
    def __init__(self, missing: set[str], available: set[str]) -> None:
        self.missing, self.available = missing, available
        super().__init__(f"missing required column(s): {sorted(missing)}. "
                         f"Available: {sorted(available)}")

class SchemaTypeError(DataValidationError):
    def __init__(self, column: str, expected: str, actual: str) -> None:
        super().__init__(f"column {column!r} should be {expected}, found {actual}")
```

A good error message says what was wrong **and** what was found instead. A validator raising these can sit inside a method chain because it returns the DataFrame unchanged when all is well:

```
def validate_orders(df: pd.DataFrame) -> pd.DataFrame:
    required = {"order_id", "amount", "order_date"}
    if missing := required - set(df.columns):
        raise MissingColumnsError(missing, set(df.columns))
    if not pd.api.types.is_numeric_dtype(df["amount"]):
        raise SchemaTypeError("amount", "numeric", str(df["amount"].dtype))
    if (df["amount"] < 0).any():
        raise DataValidationError(f"{int((df['amount'] < 0).sum())} negative amount(s)")
    return df
```

FROM THE HISTORY OF DATA SCIENCE

Pounds, newtons and a lost spacecraft

On 23 September 1999, NASA's Mars Climate Orbiter fired its engine to slip into orbit around Mars and was never heard from again. The investigation found that ground software supplied by a contractor reported thruster impulse in **pound-force seconds**, while the navigation software that consumed those numbers expected **newton-seconds**, a factor of about 4.45. For months, small trajectory corrections had been computed with the wrong scale, and the probe flew far too close to the planet. Each team's code was fine on its own. What was missing was a check **at the boundary** between them: exactly the place this chapter says to validate.

7.5 Logging

`print` is fine in a notebook. It's inadequate the moment code runs unattended, because it has no severity, no timestamp, no source, and can't be filtered or redirected without editing code.

	`print`	`logging`
Severity levels	✘	✔
Timestamps	✘	✔
Which module said it	✘	✔
Silence it without editing code	✘	✔
Send it to a file or a service	✘	✔

```
import logging, sys

logging.basicConfig(                        # configure ONCE, at the entry point
    level=logging.INFO,
    format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
    datefmt="%H:%M:%S",
    stream=sys.stdout,
    force=True,                             # override Jupyter's own setup
)
logger = logging.getLogger("pipeline.ingest")   # one logger per module
logger.info("Loaded 200,000 rows from orders.csv")
logger.warning("3 rows had a null country; defaulted to 'unknown'")
```

NOTE

Never call `basicConfig` inside a library. Configuration belongs to the application that *uses* the library, so it can decide where messages go.

Choosing a level

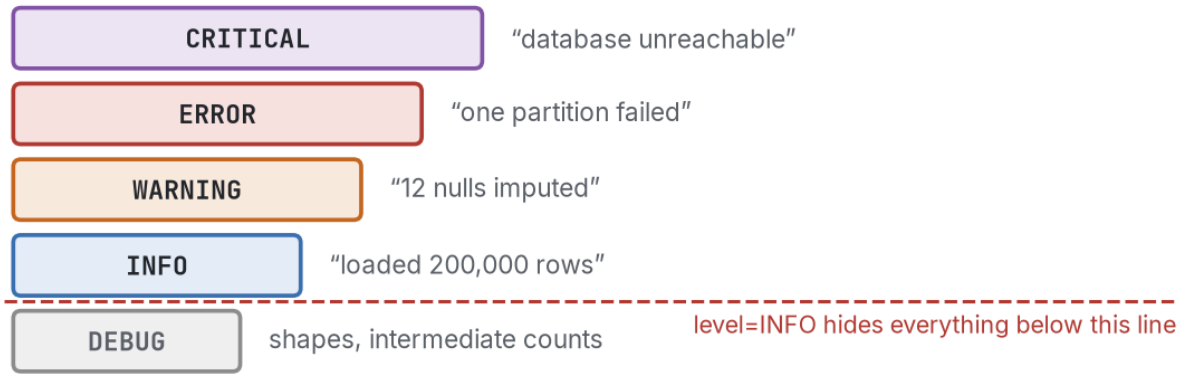


Figure 7.3 The five standard levels. Setting level=INFO hides DEBUG and keeps everything above it.

Level	Use it for	Example
`DEBUG`	Diagnostics you want only when investigating	shapes, intermediate counts
`INFO`	Confirmation that things are proceeding	"loaded 200k rows"
`WARNING`	Something unexpected but recoverable	"12 nulls imputed"
`ERROR`	An operation failed; the run may continue	"one partition failed"
`CRITICAL`	The program cannot continue	"database unreachable"

> **TIP**
> Use lazy %-style arguments: `log.info("got %d rows", n)`, not an f-string. The message is only formatted if the level is enabled, so a suppressed `DEBUG` line in a hot loop costs almost nothing.

Logging an exception with its traceback

Inside an `except` block, `logger.exception(...)` logs at `ERROR` level **and** attaches the full traceback. In a scheduled job, that's the difference between "something failed" and knowing what:

```
try:
    result = a / b
except ZeroDivisionError:
    log.exception("Division failed for a=%s b=%s", a, b)
```

7.6 Defensive validation: fail fast

The cheapest bug is the one caught at the boundary. Validate data *as it enters* a function, not three transformations later. A well-guarded function checks in three places:

```
def summarise_revenue(df: pd.DataFrame, group_col: str) -> pd.DataFrame:
    # 1. Structural checks: cheap, and they catch most real mistakes
    if group_col not in df.columns:
        raise MissingColumnsError({group_col}, set(df.columns))
    if df.empty:
        raise DataValidationError("received an empty DataFrame")
    # 2. Semantic checks: assumptions the maths depends on
    if df["revenue"].isna().any():
        raise DataValidationError("null revenue values; impute or drop first")

    result = (df.groupby(group_col, observed=True)["revenue"]
                .agg(["count", "sum", "mean"]).round(2)
                .sort_values("sum", ascending=False))

    # 3. Postcondition: verify our OWN output
    assert len(result) <= df[group_col].nunique(), "more groups than exist"
    return result
```

> WATCH OUT
>
> **`assert` is for bugs, not for data validation.** Python deletes every `assert` when run with the `-O` (optimise) flag, so an assertion is not a guarantee. Use `raise` for anything that must always be checked, including every check on incoming data. Keep `assert` for "this can only fire if *my own* code is wrong".

7.7 Testing with `pytest`

Data scientists test less than software engineers, and that's the main reason analysis bugs reach production. Yet a transformation is a *function*: input in, output out. It's the easiest thing in the world to test. Every test follows the same three beats:

```
def test_<what>_<condition>_<expected>():
    # Arrange: build the inputs
    # Act:     call the thing
    # Assert:  check the result
```

Here's a realistic feature-engineering step and its tests:

```
def add_revenue_features(df: pd.DataFrame) -> pd.DataFrame:
    if not {"amount", "quantity"}.issubset(df.columns):
        raise MissingColumnsError({"amount", "quantity"} - set(df.columns),
                                  set(df.columns))
    out = df.copy()                              # never mutate the caller's frame
    out["revenue"] = out["amount"] * out["quantity"]
    total = out["revenue"].sum()
    out["revenue_share"] = out["revenue"] / total if total else 0.0
    return out
```

```python
class TestAddRevenueFeatures:
    def test_computes_revenue_as_amount_times_quantity(self):
        df = pd.DataFrame({"amount": [10.0, 20.0], "quantity": [2, 3]})
        assert add_revenue_features(df)["revenue"].tolist() == [20.0, 60.0]

    def test_revenue_share_sums_to_one(self):
        df = pd.DataFrame({"amount": [10.0, 20.0, 5.0], "quantity": [1, 1, 1]})
        assert np.isclose(add_revenue_features(df)["revenue_share"].sum(), 1.0)

    def test_does_not_mutate_the_input(self):       # the one people forget
        df = pd.DataFrame({"amount": [10.0], "quantity": [2]})
        add_revenue_features(df)
        assert list(df.columns) == ["amount", "quantity"]

    def test_raises_on_missing_columns(self):
        with pytest.raises(MissingColumnsError, match="quantity"):
            add_revenue_features(pd.DataFrame({"amount": [10.0]}))

    def test_handles_all_zero_revenue(self):        # no division by zero
        df = pd.DataFrame({"amount": [0.0, 0.0], "quantity": [1, 2]})
        assert add_revenue_features(df)["revenue_share"].tolist() == [0.0, 0.0]

    @pytest.mark.parametrize(("amount", "quantity", "expected"), [
        (10.0, 1, 10.0), (0.0, 5, 0.0), (2.5, 4, 10.0),
        (-10.0, 2, -20.0),                          # refunds are negative
    ])
    def test_revenue_for_various_inputs(self, amount, quantity, expected):
        df = pd.DataFrame({"amount": [amount], "quantity": [quantity]})
        assert add_revenue_features(df)["revenue"].iloc[0] == expected
```

`@pytest.mark.parametrize` turns one test definition into four independent test cases. In a real project the tests live in `tests/`, and you run the whole suite by typing `pytest`. The repository's own suite runs with `.venv/bin/pytest`.

What to test in data science code

Test this	Because
Shape: row and column counts after a join	Silent row explosion from a many-to-many join is extremely common
No mutation of inputs	Chapter 2's aliasing bug, caught automatically
Edge cases: empty, one row, all-null, all-zero	Division by zero and empty aggregations
Boundaries: negative, zero, very large	Refunds, free items, outliers
Invariants: shares sum to 1, no nulls in a key	Encodes the assumption your maths relies on
Known answers: a tiny hand-computed example	Pins the definition down

> **TIP**
>
> Don't chase 100% coverage. Test the transformations whose failure would be **silent**: those are the ones that produce a plausible-looking wrong number.

7.8 Try it yourself

Exercises 7.1 to 7.5 in notebook 07; solutions in notebook 09.

1. **Narrow the catch.** A `load_and_clean(path)` wraps reading, date parsing and float conversion in one bare `except:` that returns an empty DataFrame. List three distinct failures it hides, and rewrite it so each is handled or propagated properly.
2. **Custom exception hierarchy.** Design exceptions for an ETL job: source unreachable, schema mismatch, validation failure, destination write failure, all under one base class.
3. **Instrument a function.** Add logging to `add_revenue_features`: `INFO` on entry with the row count, `WARNING` if more than 5% of amounts are null, `DEBUG` with the resulting shape.
4. **Tests first.** Before implementing `pct_change(series)`, write tests for a normal series, a zero denominator, a single element, an empty series and nulls. Then make them pass.
5. **Catch a real bug.** Write a test that fails when `customers.merge(orders, on="customer_id", how="left")` silently multiplies rows because `orders` contains duplicate IDs. (Then look up pandas' `validate=` argument.)

KEY TAKEAWAYS

- **Catch narrowly.** `except Exception:` hides bugs; a bare `except:` swallows Ctrl-C too.
- **Never return a plausible default on failure.** That's how wrong numbers reach dashboards.
- Use `raise ... from exc` to keep the causal chain.
- **Custom exceptions** let callers tell "bad data" from "broken code".
- Use `logging`, not `print`: lazy `%s` arguments, one logger per module, `logger.exception` inside `except`.
- `assert` **is for bugs;** `raise` **is for data.** Assertions vanish under `-O`.
- Test **shape, mutation, edge cases and invariants**: the failures that are silent are the ones worth testing.

CHAPTER 08

08

Project: A Pipeline You Can Trust

Every idea in the module, in one tested ETL job

Notebook 08_module_project.ipynb · About 120 minutes

IN THIS CHAPTER

- How to turn two badly behaved exports into one clean, typed Parquet table.
- Separating a thin **impure shell** (I/O) from a **pure core** (transformations).
- Counting, logging and reporting every row the pipeline drops, with the reason.
- A **quality gate** that refuses to publish output it can't vouch for.
- Testing the pieces, and proving the safety net actually fires.

Every chapter so far has handed you a tool. This one hands you a job. You've just joined the data team at a fictional retailer, Nordwind, and on your first morning two nightly exports land in your lap. Neither is well behaved.

8.1 The brief

- `orders.csv` comes from the order-management system. It's Latin-1 encoded, uses European decimal commas, has inconsistent casing, writes `"N/A"` for nulls, and contains duplicate rows whenever a retry fires.
- `customers.json` comes from the CRM, as a nested API dump.

Your job: turn these into one clean, typed, analysis-ready Parquet table, and produce a short revenue readout from it. The pipeline must be **defensive** (bad data fails loudly), **observable** (logged) and **tested**.

Chapter	What the project uses from it
2	Immutability, float comparison, string cleaning, `nan` semantics
3	`set` lookups, `defaultdict`, comprehensions, sorting
4	Pure functions, keyword-only arguments, closures, decorators
5	A frozen `@dataclass` for configuration, a mutable one for the run report

Chapter	What the project uses from it
6	`pathlib`, encodings, CSV → Parquet
7	Custom exceptions, logging, validation, `pytest`

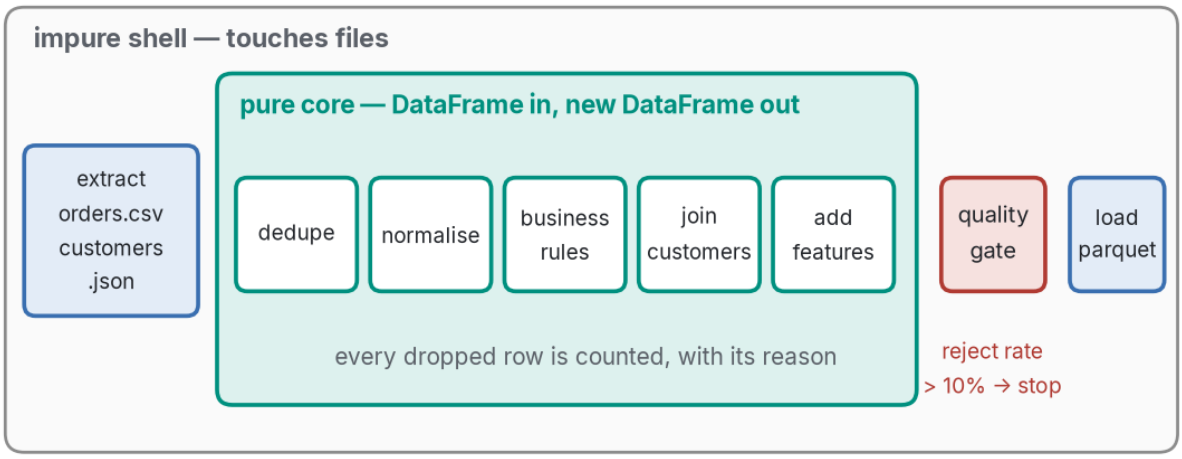


Figure 8.1 The shape of the pipeline: a pure core of transformations wrapped in a thin shell that touches files, with a quality gate before anything is published.

8.2 Step 0: generate the mess

In real life the data arrives from upstream. The notebook synthesises it, deliberately messy and reproducibly so, with a fixed seed. Seven defects are planted, each one met in production by the course's author: channels like `" PAID"` and `"organic "`, missing amounts written four different ways (`"N/A"`, `""`, `"-"`, `"null"`), decimal commas, a dozen negative quantities, duplicated rows from retries, orders pointing at a customer who doesn't exist, and a semicolon-separated, Latin-1 encoded file.

> **TIP**
> The notebook's first move is to print the **raw bytes** of the file before assuming anything about it. That ten-second habit catches encoding and separator problems that otherwise cost half an hour.

8.3 Step 1: write the contract down first

Before any transformation, state what "clean" means. Encoding the rules as a **frozen dataclass** makes them explicit, testable, and changeable in exactly one place:

```python
@dataclass(frozen=True)
class PipelineConfig:
    orders_path: Path
    customers_path: Path
    output_path: Path
    min_amount: float = 0.01                    # business rules with names,
    max_amount: float = 100_000.0               # not magic numbers scattered
    valid_channels: frozenset[str] = frozenset(  # through the code
        {"organic", "paid", "referral", "email"})
    max_reject_rate: float = 0.10               # fail the run above this

    def __post_init__(self) -> None:
        if not 0 <= self.max_reject_rate <= 1:
            raise ValueError("max_reject_rate must be between 0 and 1")
```

Alongside it sits a *mutable* dataclass, `PipelineReport`, that records what happened during a run: rows read, duplicates removed, unparseable amounts, orphan references, rows written. Its `reject_rate` is a `@property`, so it can never disagree with the counts it's computed from.

8.4 Step 2: domain exceptions

```python
class PipelineError(Exception): ...             # catch-all for callers
class SourceReadError(PipelineError): ...       # a file is missing or unreadable
class SchemaError(PipelineError): ...           # expected columns are absent
class DataQualityError(PipelineError): ...      # too much input was unusable
```

8.5 Step 3: extract (the impure shell)

The extract functions are the only ones that touch the filesystem. They read with the quirks declared **explicitly** (`encoding="latin-1"`, `sep=";"`, everything as `str`) and validate the column set immediately, raising `SchemaError` with a helpful message if it's wrong. Reading everything as text is deliberate: nothing in that file can be trusted to parse itself, so we parse it ourselves, on purpose.

8.6 Step 4: transform (the pure core)

Every function from here on is **pure**: DataFrame in, new DataFrame out, no mutation of its input, no I/O. That's what makes each one testable on its own.

Parsing European amounts

```
def parse_amount(raw: str | float | None) -> float | None:
    """'1.234,56' → 1234.56; any missing-value token → None."""
    if raw is None or (isinstance(raw, float) and np.isnan(raw)):
        return None
    text = str(raw).strip()
    if text.casefold() in {"", "n/a", "na", "-", "null", "none", "nan"}:
        return None                    # O(1) set membership, Chapter 3's habit
    try:
        return float(text.replace(".", "").replace(",", "."))
    except ValueError:
        return None                    # surface it as missing, never as 0
```

Returning `None`, not `0`, lets the caller **count** the failures instead of hiding them.

The other steps

- `deduplicate` drops exact duplicate rows and records how many.
- `normalise_columns` coerces IDs with `pd.to_numeric(..., errors="coerce")`, parses dates, trims and lower-cases text with `.str.casefold()`, and maps `parse_amount` over the amounts.
- `apply_business_rules` removes rows that break the contract, **counting each reason separately**. It uses a small closure with `nonlocal` (Chapter 4) to drop and tally in one move:

```
def drop(mask: pd.Series, reason: str) -> None:
    nonlocal out
    if n := int(mask.sum()):
        rejections[reason] = n
        out = out.loc[~mask]

drop(out["amount"].isna(), "amount missing or unparseable")
drop(out["quantity"].isna() | (out["quantity"] <= 0), "quantity not positive")
drop(~out["channel"].isin(config.valid_channels), "unknown channel")
```

- `join_customers` attaches customer attributes with a **left** join, so orders whose customer is unknown stay visible and get counted rather than silently vanishing as they would in an inner join. `validate="many_to_one"` makes pandas raise if the customer table has duplicate IDs, and a row-count check guards against the classic silent explosion:

```
merged = orders.merge(customers, left_on="customer_id", right_on="id",
                      how="left", validate="many_to_one")
if len(merged) != len(orders):
    raise DataQualityError(f"join changed the row count: "
                           f"{len(orders)} → {len(merged)}")
```

- `add_features` derives `revenue`, `order_month` and `is_weekend`, and converts repeated strings to `category` to save memory.

8.7 Step 5: orchestrate, behind a quality gate

`run_pipeline` composes the steps, logs each one, and, crucially, **refuses to publish** if too much of the input was rejected:

```
if report.reject_rate > config.max_reject_rate:
    raise DataQualityError(
        f"reject rate {report.reject_rate:.1%} exceeds the "
        f"{config.max_reject_rate:.0%} threshold; refusing to publish")
```

A pipeline that publishes a half-empty table without complaint is worse than one that fails: the failure gets fixed, but the half-empty table gets *used*.

FROM THE HISTORY OF DATA SCIENCE

Sixteen thousand cases that never arrived

In early October 2020, Public Health England discovered that 15,841 positive COVID-19 test results from the previous week had never been passed on for contact tracing. Lab results arrived as CSV files and were loaded into a spreadsheet template saved in the old `.xls` format, which holds at most 65,536 rows per sheet. When a file was bigger than that, the extra rows were simply cut off: no error, no warning, just a smaller table. Nothing in the process compared "rows read" with "rows written". A single count check, the kind of quality gate this chapter builds, would have raised the alarm on day one.

8.8 Step 6: the readout

A pipeline nobody reads from isn't finished. The notebook builds monthly revenue per acquisition channel and a per-channel table of orders, revenue and average order value, then charts both. Each channel keeps the same colour in every panel, so the colour follows the entity, not its rank. A short summary block reports total revenue, average and median order value, the largest channel's share, and the reject rate.

NOTE

The median order value sits well below the mean again: Chapter 1's right-skew, back for an encore. Reporting only the average order value would overstate what a typical order looks like.

8.9 Step 7: test it

The pipeline is only trustworthy if its parts are. The tests cover the categories from Chapter 7: known answers for `parse_amount` (`"1.234,56"`, `"89,50"`, every null token), whitespace and casing for `clean_text_column`, and **invariants** such as "revenue equals amount × quantity" and "an empty run has a reject rate of zero, not a `ZeroDivisionError`".

8.10 Step 8: read back what you wrote

The last check is to load the Parquet file exactly as a downstream consumer would and run a checklist against it: row count matches, dtypes preserved, no null revenue, every amount and quantity positive, every channel valid, no duplicate rows, and total revenue reconciling with amount × quantity. Building it, the course found a subtle trap here: text cleaned with pandas' nullable `string` dtype turned into categories that Parquet normalised back to plain `str` on read. The round trip was *almost* lossless, which is the worst kind: it passes a glance and fails an equality check. Normalising with `.astype(str).astype("category")` before writing fixes it.

8.11 Step 9: prove the safety net fires

A safety net you've never seen catch anything isn't a safety net. The notebook deliberately runs the pipeline with `max_reject_rate=0.001`, far stricter than the real reject rate, and checks that `DataQualityError` is raised and that no output file was written.

8.12 Your turn: extensions

Each of these is a real requirement you'd receive on the job.

#	Extension	Difficulty
1	**Promote** `parse_amount` and `clean_text_column` into `src/dstoolkit/cleaning.py`, with tests, and import them back.	★
2	**Stream** the CSV in chunks so the pipeline handles files bigger than RAM; check totals are unchanged.	★★
3	**Rejected-rows report**: write discarded rows to `rejected_rows.parquet` with a `rejection_reason` column.	★★
4	**Idempotency**: two runs give byte-identical output; use `atomic_write` so a crash can't corrupt the table.	★★
5	A `@monitored` **decorator** that logs each step's name, duration and row counts in and out.	★★★
6	**Declarative schema validation**: one generic validator driven by a dict of column rules. Compare with `pandera`.	★★★

8.13 What you built

A pipeline that reads badly formatted sources defensively; separates a thin impure shell from a pure core; carries its configuration in a frozen dataclass so nothing drifts mid-run; counts and logs every row it drops, with the reason; refuses to publish output it can't vouch for; is tested where the real bugs live; and writes typed, compressed Parquet that reloads exactly as written.

That's a genuinely professional shape for a data pipeline, and every later module plugs into it. Module 03 replaces the transformations with richer pandas, Module 10 adds feature engineering, Module 21 adds experiment tracking, and Module 22 puts a model behind an API.

KEY TAKEAWAYS

- Write the **contract** first, as a frozen dataclass: named rules, one place to change them.
- Keep I/O in a **thin shell**; make every transformation a **pure, testable function**.
- **Count every dropped row**, with its reason; never let rows vanish silently.
- Use **left joins** + `validate=` + **a row-count check** to make join bugs loud.
- A **quality gate** that refuses to publish is a feature, not a failure.
- **Read back what you wrote**, and **prove** your safety nets fire.

Before You Move On

You're ready for Module 02 when you can answer every question below *without looking anything up*. If one makes you hesitate, the chapter in brackets is where to go back to.

Self-assessment

1. Why does `b = a` followed by `b.append(1)` sometimes change `a`, and sometimes not? *(Chapter 2)*
2. Why is `0.1 + 0.2 == 0.3` `False`, and what should you write instead? *(Chapter 2)*
3. Why is `x in my_set` faster than `x in my_list`, and roughly by how much? *(Chapter 3)*
4. What's the difference between a list comprehension and a generator expression? *(Chapters 3–4)*
5. What does `def f(x=[])` do, and why is it a bug? *(Chapter 4)*
6. When would you use `@dataclass` rather than writing `__init__` yourself? *(Chapter 5)*
7. Why is `except Exception:` usually wrong? *(Chapter 7)*
8. Why should a pipeline use `logging` rather than `print`? *(Chapter 7)*
9. What makes Parquet a better choice than CSV for a 5 GB dataset? *(Chapter 6)*
10. Why must `assert` never be used to validate incoming data? *(Chapter 7)*

Where the exercises and solutions live

Every chapter ends with the exercises from its notebook. The full list, 29 exercises plus the project's six extensions, is in `modules/01-python-foundations/exercises/README.md`, and the worked solutions with their reasoning are in notebook `09_exercise_solutions.ipynb`.

> WATCH OUT
>
> **Attempt each exercise before opening the solution.** Reading a worked answer produces a convincing feeling of understanding that vanishes the moment you face a blank cell. Ten minutes of being stuck teaches more than an hour of reading.

Two habits will tell you whether your own answer is right, solution or not. **Write a test:** if you can't state what the correct output is, you don't understand the problem yet. And **Restart & Run All:** if your solution only works when cells run in the order you happened to click them, it doesn't work.

Further reading

Resource	Why it's worth your time
Fluent Python, 2nd ed. (Luciano Ramalho)	The definitive book on Python's object model. Its first chapters map onto Chapters 2 and 3.
PEP 8	The style guide every Python codebase assumes you've read.
PEP 484	Type hints, used throughout `src/dstoolkit/`.
The `collections` docs	`Counter` and `defaultdict` alone will save you hundreds of lines.
TimeComplexity wiki	The cost of every built-in operation, in one table.

Sources for the Stories

The history boxes in this book simplify, as stories must, but none of them is invented. These are the sources behind each one, in the order they appear, so that you can read the full versions and check the details for yourself.

Chapter 1

- **The title "data scientist".** T. H. Davenport and D. J. Patil, "Data Scientist: The Sexiest Job of the 21st Century", *Harvard Business Review*, October 2012.
- **John Snow and Broad Street.** J. Snow, *On the Mode of Communication of Cholera*, 2nd ed., London: John Churchill, 1855. For a readable modern account: S. Johnson, *The Ghost Map*, Riverhead, 2006.
- **Reinhart and Rogoff.** C. M. Reinhart and K. S. Rogoff, "Growth in a Time of Debt", *American Economic Review* 100(2), 2010; T. Herndon, M. Ash and R. Pollin, "Does High Public Debt Consistently Stifle Economic Growth? A Critique of Reinhart and Rogoff", *Cambridge Journal of Economics* 38(2), 2014 (working paper 2013).
- **"Student".** Student [W. S. Gosset], "The Probable Error of a Mean", *Biometrika* 6(1), 1908.

Chapter 2

- **Gangnam Style and the 32-bit counter.** YouTube's own announcement, early December 2014, widely reported by the international press at the time.
- **The Vancouver Stock Exchange index.** Reported in *The Wall Street Journal*, November 1983, when the exchange published the recalculated index; since then a standard example in courses on numerical computing.
- **Python's origins.** G. van Rossum, foreword to *Programming Python*, 1st ed., O'Reilly, 1996; and the Python documentation's "General Python FAQ".

Chapter 3

- **Why numbering should start at zero.** E. W. Dijkstra, EWD831, 11 August 1982. Available from the E. W. Dijkstra Archive, University of Texas at Austin.
- **Hashing at IBM.** D. E. Knuth, *The Art of Computer Programming*, Vol. 3: *Sorting and Searching*, 2nd ed., Addison-Wesley, 1998, section 6.4 (history of hashing, crediting H. P. Luhn's January 1953 IBM memorandum).
- **The Federalist Papers.** F. Mosteller and D. L. Wallace, *Inference and Disputed Authorship: The Federalist*, Addison-Wesley, 1964.
- **Timsort.** T. Peters, "listsort.txt", in the CPython source tree, 2002.

Chapter 4

- **Memo functions.** D. Michie, "'Memo' Functions and Machine Learning", *Nature* 218, 1968.
- **The lambda calculus.** A. Church, "An Unsolvable Problem of Elementary Number Theory", *American Journal of Mathematics* 58(2), 1936.
- **MapReduce.** J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters", *OSDI*, 2004.

Chapter 5

- **Simula.** O.-J. Dahl and K. Nygaard, "SIMULA: An ALGOL-Based Simulation Language", *Communications of the ACM* 9(9), 1966; ACM A. M. Turing Award citation, 2001.
- **Composition over inheritance.** E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- **scikit-learn.** F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research* 12, 2011; L. Buitinck et al., "API design for machine learning software: experiences from the scikit-learn project", ECML PKDD Workshop, 2013.

Chapter 6

- **CSV.** Y. Shafranovich, RFC 4180, "Common Format and MIME Type for Comma-Separated Values (CSV) Files", IETF, October 2005 (Informational).
- **Excel and gene names.** M. Ziemann, Y. Eren and A. El-Osta, "Gene name errors are widespread in the scientific literature", *Genome Biology* 17, 2016; E. A. Bruford et al., "Guidelines for human gene nomenclature", *Nature Genetics* 52, 2020.
- **JSON.** D. Crockford, json.org; ECMA-404, *The JSON Data Interchange Syntax*, 2013.
- **Dremel and Parquet.** S. Melnik et al., "Dremel: Interactive Analysis of Web-Scale Datasets", *Proceedings of the VLDB Endowment* 3(1), 2010; Apache Parquet project history, parquet.apache.org.

Chapter 7

- **The moth.** Logbook of the Harvard Mark II, 9 September 1947, Smithsonian National Museum of American History.
- **Mars Climate Orbiter.** NASA, *Mars Climate Orbiter Mishap Investigation Board Phase I Report*, 10 November 1999.

Chapter 8

- **Public Health England's lost test results.** Public Health England statement of 4 October 2020 and the Health Secretary's statement to Parliament on 5 October 2020; reported widely, including by BBC News, at the time.